AD23532 PRINCIPLES OF DATA SCIENCE · MINI PROJECT · LEARNING GUIDE

Distraction Pattern Detector

Build the whole project yourself, from zero: Python, data cleaning, feature engineering, EDA, six machine-learning models, evaluation, tuning, a live web app and GitHub deployment. Every concept and every line of code explained.

```
problem → data → cleaning → features → EDA → 6 models → tuning → selection → web app → GitHub Pages
```

Jeshwanth · B.Tech Artificial Intelligence and Data Science · Rajalakshmi Engineering College · Semester V, 2026-27

Code: github.com/Jeshwanth-tkd/Distraction-Pattern-Detector

Live app: jeshwanth-tkd.github.io/Distraction-Pattern-Detector

Contents

1 The project in one picture

2 Setting up your computer

3 Just enough Python and pandas

4 Defining the problem (Review 1)

5 Where the data comes from

6 Data quality audit (Review 2, part 1)

7 Cleaning the data (Review 2, part 2)

8 Creating features (Review 2, part 3)

9 Choosing features and combining tables

10 Exploring the data (EDA)

11 How machine learning works (the core ideas)

12 Measuring a model: the metrics

13 The six models, one by one

14 Tuning the models

15 Comparing models and choosing the winner

16 Turning the model into a website

17 Putting it on GitHub and going live

18 Preparing for your reviews

19 Glossary

START HERE

How to use this guide

This guide teaches you the whole mini project from zero. You do not need to know Python, statistics or machine learning before you start. Every idea is explained the first time it appears, every piece of code is shown and explained line by line, and every chapter ends with a short check so you know you understood it.

Read it in order. Each chapter builds on the one before. The chapters follow the same order as your three reviews:

Part	Chapters	What you learn	Review
A. Foundations	1–3	The project in one picture, setting up your computer, just enough Python and pandas	–
B. Problem and data	4–5	Defining the problem, the project charter, how the dataset is built	Review 1
C. Cleaning and exploring	6–10	Finding and fixing bad data, creating features, exploring with charts and statistics	Review 2
D. Machine learning	11–15	How models learn, how to measure them, the six models, tuning, choosing the winner	Review 3
E. Product	16–17	Turning the model into a website and putting it on GitHub	Review 3
F. Review prep	18–19	Viva questions with answers, glossary	All

How to practise. Keep the project open next to this guide (`Documents\Distraction-Pattern-Detector`, or download it from `github.com/Jeshwanth-tkd/Distraction-Pattern-Detector`). When a chapter shows code, type it yourself in a notebook cell and run it. Typing it yourself is how it sticks.

Boxes in this guide mean:

Idea. A concept explained in plain words.

Try it. Something to type and run.

Common mistake. A trap beginners fall into.

Viva. A question your examiner may ask, with a short answer.

CHAPTER 1

The project in one picture

1.1 The question we answer

Imagine your phone could tell you at the end of the day: "*Today was not a normal day for you. You switched apps three times more than usual, you did half your usual study time, and you were on your phone until 2 a.m.*"

That is exactly what this project builds. For any one day of a person's phone and computer use, it predicts one of three labels:

Label	Meaning
Normal	The day looks like this person's usual days
Increased Distraction	Somewhat more distracted than usual
Unusually Distracted	Much more distracted than usual

1.2 The one idea that makes the project work

Compare a person with themselves, not with everyone else. Six hours of screen time is a normal day for a heavy user and a very unusual day for a light user. So instead of asking "is 6 hours a lot?", we ask "is 6 hours a lot *for this person*, compared with their own last 14 days?"

This "compare with your own past" idea is called a **personal baseline**. You will see it in almost every chapter. When your examiner asks "what is special about your project?", this is the answer.

1.3 The pipeline

Every data science project follows roughly the same path. Here is ours:

```
 1. Problem        What are we predicting, and why does it matter?
       |
 2. Data           40 people × 120 days of usage logs (with deliberate errors)
       |
 3. Cleaning       Fix missing values, duplicates, wrong formats, impossible values
       |
 4. Features       Build new columns: ratios and "how far from my normal" scores
       |
 5. EDA            Charts and statistics to understand the data
       |
 6. Models         Train 6 different methods, measure each one fairly
       |
 7. Tuning         Find the best settings for each method
       |
 8. Selection      Pick the best model and prove it with numbers
       |
 9. Product        Put the model inside a website anyone can use
       |
10. Deploy         Upload to GitHub, publish the website with GitHub Pages
```

1.4 The final result

On one month of data the model had never seen (April), the chosen model, a tuned **Logistic Regression**, got:

Measure	Value	Meaning in plain words
Accuracy	0.812	It gave the right label on 81 of every 100 days
Macro-F1	0.763	A fair average score across all three labels
F1 for *Unusually Distracted*	0.786	It is good at catching the worst days
ROC-AUC	0.898	It ranks distracted days above normal days 90% of the time

You will learn exactly what each of these means in Chapter 12.

1.5 The files in the project

```
Distraction-Pattern-Detector/
├── data/
│   ├── raw_digital_activity.csv      the messy data (4,885 rows)
│   └── clean_digital_activity.csv    the cleaned data (4,800 rows)
├── notebooks/
│   ├── Distraction_Detection.ipynb   THE MAIN FILE: all analysis with outputs
│   └── distraction_project.py        the same notebook as a plain .py script
├── src/
│   ├── generate_dataset.py           creates the raw data
│   ├── run_notebook.py               runs the .py and saves the .ipynb
│   └── build_app.py                  builds the website from the model
├── models/final_model.pkl            the trained model, saved
├── figures/                          the 18 charts
├── app/                              website source files
├── docs/index.html                   the published website
├── reports/Technical_Report.md       the written report
└── README.md                         the front page on GitHub
```

✓ **Check yourself.** (1) What are the three labels? (2) What is a personal baseline, in one sentence? (3) Which file holds all the analysis?

CHAPTER 2

Setting up your computer

You need three things: **Python** (the programming language), a few **libraries** (ready-made code other people wrote), and a place to **write and run** code.

2.1 Option A: Google Colab (easiest, nothing to install)

Colab runs Python in your browser, on Google's computers.

1. Go to `colab.research.google.com` and sign in with a Google account.
2. Click **File** → **Upload notebook** and choose `notebooks/Distraction_Detection.ipynb`.
3. On the left, click the folder icon and upload the `data` folder's two CSV files into a folder called `data`.
4. Run cells with **Shift + Enter**.

> **Common mistake.** In Colab the notebook looks for data in `../data/`. If you upload the CSVs next to the notebook instead, change `DATA = Path("../data")` to `DATA = Path(".")` in the first code cell.

2.2 Option B: Your own computer (recommended for the review)

1. **Install Python.** Download Python 3.11 or newer from `python.org`. During install on Windows, tick **"Add Python to PATH"**. This lets you type `python` in a terminal.
2. **Install VS Code** from `code.visualstudio.com`, then inside VS Code install the **Python** and **Jupyter** extensions (Extensions icon on the left, search, Install).
3. **Open a terminal** (in VS Code: Terminal → New Terminal) and install the libraries:

```
pip install pandas numpy scikit-learn scipy matplotlib seaborn jupyter
```

Finally, **open the project folder** in VS Code (File → Open Folder), then open `notebooks/Distraction_Detection.ipynb` and click **Run All**.

2.3 What each library does

Library	What it is for	Used in our project for
pandas	Tables of data (like Excel inside Python)	Reading CSV files, cleaning, grouping
NumPy	Fast maths on lists of numbers	Random numbers, arrays, maths
scikit-learn (`sklearn`)	Machine learning	All six models, metrics, cross-validation
SciPy	Statistics	Kruskal-Wallis and McNemar tests
Matplotlib	Drawing charts	Every figure
Seaborn	Prettier charts on top of Matplotlib	Box plots, heatmaps, histograms

2.4 Running the whole project from scratch

From inside the project folder, three commands rebuild everything:

```
python src/generate_dataset.py
python src/run_notebook.py notebooks/distraction_project.py Distraction_Detection.ipynb
python src/build_app.py
```

The first makes the data, the second runs all the analysis (about 5 minutes) and saves the notebook with its outputs, the third builds the website.

Try it. Run just the first command. Then open `data/raw_digital_activity.csv` in Excel and scroll through it. You should see 4,885 rows. Look for oddities: dates written differently, a `9999`, labels in capitals. You will fix all of these in Chapter 7.

✓ **Check yourself.** Which library would you use to (a) read a CSV, (b) train a Random Forest, (c) draw a histogram?

CHAPTER 3

Just enough Python and pandas

This chapter teaches only what the project uses. If you already know Python, skim it.

3.1 Python basics in 10 minutes

Variables store values. **Types** say what kind of value.

```
users = 40                  # int: whole number
screen_hrs = 5.5            # float: decimal number
label = "Normal"            # str: text, in quotes
is_weekend = True           # bool: True or False
```

Lists hold several values in order. **Dictionaries** hold *key* → *value* pairs.

```
classes = ["Normal", "Increased Distraction", "Unusually Distracted"]
print(classes[0])           # Normal   (counting starts at 0!)

colors = {"Normal": "green", "Unusually Distracted": "red"}
print(colors["Normal"])     # green
```

Loops repeat work. **if** makes decisions.

```
for c in classes:
    print(c)

score = 2.3
if score >= 2:
    print("Unusually Distracted")
elif score >= 1:
    print("Increased Distraction")
else:
    print("Normal")
```

That last snippet is literally our first model (Chapter 13)!

Functions package code so you can reuse it.

```
def label_from_score(score):
    if score >= 2:
        return "Unusually Distracted"
    elif score >= 1:
        return "Increased Distraction"
    return "Normal"

label_from_score(1.4)       # 'Increased Distraction'
```

f-strings put values inside text: `f"Accuracy = {0.8123:.3f}"` gives `Accuracy = 0.812`. The `:.3f` means "3 decimal places".

Imports load a library. `import pandas as pd` means "load pandas and call it `pd` for short".

List comprehension is a short way to build a list:

```
squares = [x * x for x in range(5)]            # [0, 1, 4, 9, 16]
z_cols = [c for c in columns if c.startswith("z_")]   # keep only names starting with z_
```

3.2 NumPy in 3 minutes

NumPy works on whole arrays at once, without loops.

```
import numpy as np
a = np.array([10, 20, 30])
a * 2              # array([20, 40, 60])
a.mean()           # 20.0
np.where(a > 15, "big", "small")   # array(['small', 'big', 'big'])
```

`np.where(condition, if_true, if_false)` is used a lot in the project to turn scores into labels.

3.3 pandas: the most important tool

A **DataFrame** is a table. Each column has a name, each row has an index.

```
import pandas as pd
df = pd.read_csv("data/clean_digital_activity.csv")
df.head()          # first 5 rows
df.shape           # (4800, 14) → rows, columns
df.info()          # column names, types, missing counts
df.describe()      # count, mean, std, min, quartiles, max for numbers
```

Selecting

```
df["app_switches"]                          # one column (a "Series")
df[["user_id", "app_switches"]]             # several columns
df[df["label"] == "Normal"]                 # only rows where label is Normal
df[(df.user_id == "U003") & (df.is_weekend == 1)]   # two conditions: use & and brackets
```

Counting and summarising

```
df["label"].value_counts()                  # how many of each label
df["screen_time_hrs"].mean()
df.isna().sum()                             # missing values per column
df.duplicated().sum()                       # number of duplicate rows
```

groupby: the key to personal baselines

`groupby` splits the table into groups, applies something to each group, and joins the results.

```
df.groupby("user_id")["screen_time_hrs"].mean()      # each person's average screen time
```

```
user_id
U001    4.71
U002    6.02
U003    3.95
...
```

There are two flavours you will see:

- `.agg(...)` / `.mean()` gives **one row per group** (40 rows for 40 users).
- `.transform(...)` gives **one value per original row** (4,800 rows), so you can put the result back as a new column. We use `transform` to give every day its own user's median or baseline.

```
df["user_median_social"] = df.groupby("user_id")["social_media_min"].transform("median")
```

shift and rolling: looking at previous days

```
s = pd.Series([10, 20, 30, 40])
s.shift(1)                  # [NaN, 10, 20, 30]   → "yesterday's value"
s.rolling(2).mean()         # [NaN, 15, 25, 35]   → average of the last 2 values
```

These two together build the personal baseline in Chapter 8.

Other operations used

Code	What it does
`df.drop_duplicates(subset=[...])`	Remove repeated rows
`df.fillna(value)`	Replace missing values
`pd.to_datetime(col, format=...)`	Turn text into real dates
`df.merge(other, on="user_id")`	Join two tables on a shared column (like VLOOKUP)
`df.pivot_table(...)`	Reshape: rows by one thing, columns by another
`df.sort_values("date")`	Sort rows
`df.to_csv("file.csv")`	Save the table

Common mistake. `df[df.a > 1 and df.b < 2]` crashes. With pandas use `&` (and), `|` (or), `~` (not), and put each condition in brackets: `df[(df.a > 1) & (df.b < 2)]`.

Try it. Load the clean CSV and find: (1) how many days each label has, (2) the user with the highest average `app_switches`, (3) the average `night_usage_min` on weekends versus weekdays. Hint for (3): `df.groupby("is_weekend")` does not exist in the clean CSV yet, so first make it: `df["is_weekend"] = pd.to_datetime(df.date).dt.dayofweek >= 5`.

✓ **Check yourself.** What is the difference between `groupby().mean()` and `groupby().transform("mean")`? What does `shift(1)` do?

CHAPTER 4

Defining the problem (Review 1)

Before any code, a data scientist writes down what they are solving. Review 1 asks for exactly this.

4.1 Problem statement

> *Given one day of a person's digital activity, predict whether that day is* ***Normal****, shows* ***Increased Distraction****, or is* ***Unusually Distracted****, compared with that person's own recent behaviour.*

Why it matters. Students lose study time to constant app switching. Professionals see productive hours shrink before deadlines. Phone wellbeing apps only show raw hours, which do not tell you whether today was unusual *for you*.

4.2 What kind of problem is it?

Data science problems come in a few types:

Type	Question it answers	Example
Classification	Which group does this belong to?	Normal / Increased / Unusual
Regression	What number will it be?	How many hours tomorrow?
Clustering	Which items are naturally similar?	Groups of similar days
Anomaly detection	Which items are strange?	Days unlike the rest

Ours is mainly **classification** with **three classes** (called *multi-class*). We also try clustering and anomaly detection as alternatives, because in real life people rarely label their days, and it is useful to know how well we could do without labels.

Supervised vs unsupervised. *Supervised* learning learns from examples that already have the answer (the label). *Unsupervised* learning gets no answers and must find patterns on its own. Logistic Regression, Decision Tree and Random Forest are supervised. K-Means and Isolation Forest are unsupervised.

4.3 Objectives

1. Build a personal baseline for each user from their own recent history.
2. Create features that measure how much a day differs from that baseline.
3. Train and fairly compare at least six methods.
4. Choose the best one with evidence, and put it in a working web app.

4.4 Project charter

A charter is a one-page agreement about the project.

Item	Our answer
Mission	Detect the change, understand the pattern, give useful insight
Stakeholders	Students, working professionals, digital-wellbeing app makers, mentors
In scope	Daily usage counts, three-class prediction, a web demo
Out of scope	Medical diagnosis, reading message content, minute-by-minute tracking
Outcome	Clean data, EDA, tuned model, deployed app, report

Say this clearly in your review. The project detects *changes in usage pattern*. It does **not** diagnose addiction or any medical condition. Examiners like to see that you know the limits of your work.

4.5 Timeline (6 weeks)

Week	Work	Output
1	Set up tools, read related work	Charter (Review 1)
2	Build or collect the dataset	Raw data
3	Cleaning	Clean data
4	EDA and features	EDA report (Review 2)
5	Models and tuning	Comparison tables
6	Final model, website, report	Deployed app (Review 3)

Q: Why classification and not regression? We want a decision (normal or not, and how bad), not a number of hours. Three ordered categories are easier to act on than a raw number.

CHAPTER 5

Where the data comes from

5.1 Why the data is simulated

Real per-app phone logs with a label for every day are private. No public dataset has 40 people's daily social, gaming, productive time, app switches and night use *with* a distraction label for each day. So the project **generates** realistic data with a program, `src/generate_dataset.py`.

This is a normal, accepted practice called **synthetic data** or **simulation**, as long as you say so openly. The notebook and the report both say it.

Q: Is your data real? No, it is simulated, because real labelled per-app logs are private. The generator copies patterns reported in screen-time studies: every person has their own normal level, distraction lasts for a few days once it starts, weekends change behaviour, and some labels are wrong (like real self-reports). If I had real consented data, the same code would run on it unchanged.

5.2 The ideas inside the generator

1. Every person has their own normal

Each of the 40 users gets their own typical values: typical screen time, typical mix of social / gaming / productive / other time, typical app switches, typical night use. Students are a little heavier users than professionals. This is what makes the personal baseline necessary.

2. A hidden "state" for each day, using a Markov chain

Each day the person is secretly in one of the three states. Tomorrow's state depends on today's. This is called a **Markov chain**.

Markov chain, in plain words. A rule for "what happens next depends only on what is happening now." If you were distracted today, you are more likely to be distracted tomorrow too, just like a bad week tends to stay bad for a few days.

The rule is a table of probabilities. Each row is today's state, each column is tomorrow's:

Today ↓ / Tomorrow →	Normal	Increased	Unusual
Normal	0.82	0.14	0.04
Increased	0.45	0.40	0.15
Unusual	0.30	0.35	0.35

So a normal day is followed by another normal day 82% of the time. Each row adds up to 1. Weekends use a second table where distraction is a bit more likely.

3. The state changes behaviour

In each state, the person's usual values are multiplied:

Signal	Normal	Increased	Unusual
Social media	×1.00	×1.55	×2.50
Gaming	×1.00	×1.45	×2.30
Productive time	×1.00	×0.80	×0.52
App switches	×1.00	×1.35	×2.00
Night use	×1.00	×1.50	×2.40

So on an *Unusually Distracted* day, social media is 2.5 times the person's own normal and productive time is about half.

4. Random day-to-day noise

Real days are never identical. Every value is multiplied by a random number from a **log-normal distribution**: usually close to 1, sometimes bigger, never negative. That gives realistic right-skewed data (many ordinary days, a few very heavy ones).

5. Imperfect labels

7% of labels are moved to a neighbouring class, because people's own reports of "was I distracted?" are never perfect. This means **no model can ever reach 100%**, which is realistic.

6. Deliberate data errors

Finally the generator damages the data in the ways real exported logs get damaged, so you have real cleaning to do. Chapter 6 lists every one.

5.3 The code, piece by piece

Random numbers with a fixed seed. A seed makes the "random" numbers the same every time you run, so your results can be repeated. This is called **reproducibility**.

```
rng = np.random.default_rng(42)       # 42 = the seed
rng.normal(5.0, 1.1)                  # a random number around 5.0
rng.choice(3, p=[0.82, 0.14, 0.04])   # pick 0, 1 or 2 with these probabilities
```

Making a user (simplified):

```
base_screen = rng.normal(5.0, 1.1)              # typical hours per day
shares = rng.dirichlet([3.0, 1.6, 4.0, 3.0])    # split into social/gaming/productive/other
```

A *Dirichlet* draw gives four positive numbers that add up to 1, a random "recipe" for how this person splits their time.

Simulating one day:

```
state = rng.choice(3, p=T[state])      # tomorrow's state from today's row of the table

def ln(x):                             # add day-to-day noise
    return x * rng.lognormal(0, vol)

social = ln(s * MULT["social"][state] * wk_social)
```

Read the last line as: *this person's usual social minutes* × *the state multiplier* × *a weekend boost* × *random noise*.

Damaging the data (two examples):

```
df.loc[pick(0.025), "social_media_min"] = np.nan    # 2.5% of rows lose this value
df.loc[pick(14), "app_switches"] = 9999             # 14 rows get a sensor glitch
```

`df.loc[rows, column] = value` means "in these rows of this column, put this value". `np.nan` means "missing" (Not a Number).

The final file has **4,885 rows and 14 columns**: 40 users × 120 days = 4,800 real days, plus 85 duplicates.

5.4 The columns

Column	Type	Meaning
record_id	number	Row id
user_id	text	U001 … U040
user_type	text	Student or Professional
date	date	The day
screen_time_hrs	decimal	Total screen time in hours
social_media_min	decimal	Minutes on social apps
gaming_min	decimal	Minutes gaming
productive_min	decimal	Minutes on work/study apps
other_min	decimal	Everything else
app_switches	whole number	How many times the user changed apps
night_usage_min	decimal	Minutes used between 23:00 and 05:00
phone_unlocks	whole number	Unlocks in the day
notifications	whole number	Notifications received
label	text	**The target**: what we want to predict

Feature vs target. *Features* are the inputs the model looks at (screen time, switches…). The *target* (or *label*) is the answer we want it to predict.

✓ **Check yourself.** (1) Why does the generator use a seed? (2) In the Markov table, what is the chance that an Unusual day is followed by a Normal day? (3) Why can no model reach 100% accuracy on this data?

CHAPTER 6

Data quality audit (Review 2, part 1)

Before fixing data you must **find and document** what is wrong. This is the "data quality assessment" deliverable.

6.1 Looking at the raw data

```
raw = pd.read_csv(DATA / "raw_digital_activity.csv")
raw.shape          # (4885, 14)
raw.info()         # shows which columns have missing values
raw.describe().T   # min/max reveal impossible values
```

`describe()` immediately shows problems: `gaming_min` has a minimum of **−35** (negative minutes are impossible), `screen_time_hrs` has a maximum of **29.85** (a day has 24 hours), `app_switches` has a maximum of **9999**.

6.2 The audit code

The notebook checks each kind of problem and puts the counts into one table:

```
issues = []
for c, n in raw.isna().sum().items():           # 1. missing values per column
    if n:
        issues.append(("Missing values", c, n))

dup = raw.duplicated(subset=raw.columns.drop("record_id")).sum()   # 2. duplicates
issues.append(("Exact duplicate records", "all columns", dup))

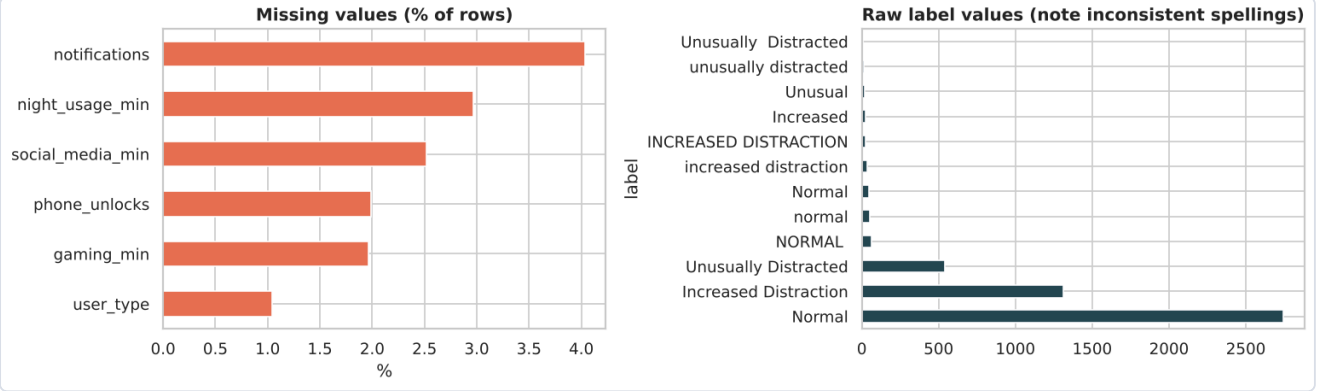
issues.append(("Inconsistent label spellings", "label",
               (~raw["label"].isin(CLASSES)).sum()))               # 3. bad spellings
issues.append(("Non-ISO date format", "date",
               raw["date"].str.contains("/").sum()))               # 4. dd/mm/yyyy dates
issues.append(("Negative duration", "gaming_min", (raw["gaming_min"] < 0).sum()))
issues.append(("Screen time > 24 h", "screen_time_hrs", (raw["screen_time_hrs"] >
24).sum()))
issues.append(("Sensor glitch 9999", "app_switches", (raw["app_switches"] == 9999).sum()))
```

Small things to notice:

- `.isin(list)` asks "is the value one of these?"; `~` flips it to "is NOT one of these".
- `.str.contains("/")` checks text. ISO dates look like `2024-03-10`; the bad ones look like `10/03/2024`.
- `.sum()` on True/False counts the Trues.
- We drop `record_id` before checking duplicates, because a copied row gets a new id.

6.3 What the audit found

Issue	Column	Rows
Missing values	user_type 51, social 123, gaming 96, night 145, unlocks 97, notifications 197	1–4% each
Exact duplicate records	all	85
Inconsistent label spellings (`normal`, `INCREASED DISTRACTION`, `Unusual`…)	label	290
Inconsistent user_type spellings (`STUDENT`, `student`)	user_type	190
Dates as dd/mm/yyyy	date	242
Negative minutes	gaming_min	13
Screen time over 24 h	screen_time_hrs	8
Night use over the 6-hour night window	night_usage_min	10
Sensor glitch 9999	app_switches	14
Total does not equal the sum of its parts	screen_time_hrs	21



Missing values and the raw label spellings

6.4 Outliers: errors or real?

An **outlier** is a value far from the rest. The **IQR rule** is a common test:

> **IQR rule.** Sort the values. Q1 is the value 25% of the way up, Q3 is 75% of the way up. IQR = Q3 − Q1 (the width of the middle half). Anything below Q1 − 1.5×IQR or above Q3 + 1.5×IQR is called an outlier.

Example: if Q1 = 40 and Q3 = 120 minutes of social media, IQR = 80, so the upper fence is 120 + 1.5 × 80 = 240. A 300-minute day is an "outlier".

But here is the key judgement: **most of these outliers are real distracted days**, which are exactly what we want to predict. Deleting them would delete the signal. So we only remove values that are *impossible* (negative, over 24 hours, 9999), not values that are merely *large*.

Q: Why didn't you remove all outliers? Because in this problem the extreme days are the distracted days we are trying to detect. I removed only impossible values (negative minutes, more than 24 hours, the 9999 glitch) and kept genuine heavy-use days.

✓ **Check yourself.** Which three `describe()` numbers tell you something is wrong? What does `~raw["label"].isin(CLASSES)` find?

CHAPTER 7

Cleaning the data (Review 2, part 2)

Now we fix every issue, explain why each fix is the right one, and then **prove** the data is clean.

7.1 Step 1: fix spellings

```
label_map = {"normal": "Normal",
             "increased": "Increased Distraction",
             "increased distraction": "Increased Distraction",
             "unusual": "Unusually Distracted",
             "unusually distracted": "Unusually Distracted"}

df["label"] = (df["label"].str.strip()                        # remove spaces at the ends
                          .str.lower()                         # make lowercase
                          .str.replace(r"\s+", " ", regex=True)  # many spaces → one space
                          .map(label_map))                     # map to the official name
df["user_type"] = df["user_type"].str.strip().str.title()      # "STUDENT " → "Student"
```

The trick: first make every spelling the same shape (trimmed, lowercase, single spaces), then a small dictionary maps each shape to the official label. `.title()` capitalises the first letter of each word.

7.2 Step 2: unify the dates

```
iso = pd.to_datetime(df["date"], format="%Y-%m-%d", errors="coerce")
dmy = pd.to_datetime(df["date"], format="%d/%m/%Y", errors="coerce")
df["date"] = iso.fillna(dmy)
```

We try to read every date as `year-month-day`. Dates in the other format fail and become missing (`errors="coerce"` means "don't crash, just give NaN"). We then try them as `day/month/year` and fill the gaps. Result: one clean date column, no rows lost.

7.3 Step 3: remove duplicates

```
df = df.drop_duplicates(subset=df.columns.drop("record_id"))
df = df.sort_values("record_id").drop_duplicates(subset=["user_id", "date"], keep="first")
```

A person can only have **one** log per day. So the **natural key** is `user_id + date`. Any second row with the same pair is a duplicate sync. 85 rows removed.

7.4 Step 4: impossible values become missing

```
df.loc[df["gaming_min"] < 0, "gaming_min"] = np.nan
df.loc[df["app_switches"] == 9999, "app_switches"] = np.nan
df.loc[df["night_usage_min"] > 360, "night_usage_min"] = np.nan
```

Why not just change 9999 to the maximum? That would invent a fake extreme day. The honest move is to say "we don't know this value" (missing) and then fill it sensibly in the next step.

7.5 Step 5: fill missing values with the person's own median

```
fill_cols = ["social_media_min", "gaming_min", "night_usage_min",
             "app_switches", "phone_unlocks", "notifications"]
df[fill_cols] = df.groupby("user_id")[fill_cols].transform(lambda s: s.fillna(s.median()))
```

Read it as: *for each user separately, fill each missing value with that user's own median.*

Two choices to defend:

- **Why median, not mean?** The data is skewed: a few huge days pull the mean up. The median (the middle value) is not affected by them. This is called being **robust to outliers**.
- **Why the person's own median?** A heavy user's missing value should be filled with a heavy-user number. Using everyone's median would pull heavy users down and light users up.

`user_type` never changes for a person, so its gaps are filled from that person's other rows (their most common value, the *mode*).

`lambda s: ...` is a tiny one-line function with no name. Here `s` is one user's column.

7.6 Step 6: recompute total screen time

```
df["screen_time_hrs"] = (df[["social_media_min", "gaming_min",
                             "productive_min", "other_min"]].sum(axis=1) / 60).round(2)
```

Total screen time must equal the sum of its four parts (in hours). Recomputing it fixes the 8 rows over 24 hours and the 21 mismatched rows in one go. `axis=1` means "add across the columns of each row".

7.7 Step 7: validate

Never just *assume* cleaning worked. Write tests:

```
checks = {
    "no missing values":           df.isna().sum().sum() == 0,
    "one row per user per day":    not df.duplicated(["user_id", "date"]).any(),
    "40 users x 120 days":         len(df) == 40 * 120,
    "labels in the 3 valid classes": df["label"].isin(CLASSES).all(),
    "all durations >= 0":          (df[num_cols] >= 0).all().all(),
    "screen time <= 24 h":         (df["screen_time_hrs"] <= 24).all(),
}
for name, ok in checks.items():
    print("PASS" if ok else "FAIL", name)
assert all(checks.values())     # stops the notebook if anything failed
```

All checks pass, and we save `clean_digital_activity.csv` with exactly **4,800 rows**.

Try it. Before Step 5, run `df.isna().sum()`. After Step 5, run it again. Then pick one user, say `U010`, and check that their filled values equal `df[df.user_id=="U010"]["social_media_min"].median()`.

Q: Which imputation did you use and why? Per-user median. Median because the data is right-skewed, so the median is robust to heavy days. Per-user because each person has a different normal level, so a population value would distort heavy and light users.

✓ **Check yourself.** (1) What is the natural key of this dataset? (2) Why is 9999 set to missing instead of being clipped? (3) What does `assert` do?

CHAPTER 8

Creating features (Review 2, part 3)

Feature engineering means creating new columns that make the pattern easier for a model to see. This chapter is the heart of the project.

8.1 Simple encodings

Models only understand numbers, so text must be turned into numbers. This is called **encoding**.

```
df["day_of_week"] = df["date"].dt.dayofweek                # Monday=0 … Sunday=6
df["is_weekend"]  = (df["day_of_week"] >= 5).astype(int)     # 1 on Sat/Sun, else 0
df["is_student"]  = (df["user_type"] == "Student").astype(int)
df["label_code"]  = df["label"].map({"Normal": 0, "Increased Distraction": 1,
                                     "Unusually Distracted": 2})
```

- `is_student` is **binary encoding**: two categories become 0/1.
- `label_code` is **ordinal encoding**: the three labels have a natural order (more and more distracted), so 0, 1, 2 keeps that order.

8.2 Ratio features

Raw minutes depend on how much someone uses their phone at all. Ratios remove that effect.

```
total_min = df["screen_time_hrs"] * 60
df["distraction_ratio"]  = (df["social_media_min"] + df["gaming_min"]) / total_min
df["productive_ratio"]   = df["productive_min"] / total_min
df["switch_rate_per_hr"] = df["app_switches"] / df["screen_time_hrs"]
df["night_usage_ratio"]  = df["night_usage_min"] / total_min
df["unlocks_per_hr"]     = df["phone_unlocks"] / df["screen_time_hrs"]
```

Example: 120 minutes of social plus 30 of gaming out of 300 total minutes gives a distraction ratio of 150 / 300 = **0.5**: half the day's screen time was distraction.

8.3 The personal baseline and z-scores

What a z-score is

> **z-score = (today – my usual) ÷ my usual spread.** It answers "how many *standard deviations* away from my normal is today?" A z-score of 0 means a perfectly typical day. +1 means noticeably higher than usual. +2 means much higher; only about 2–3% of ordinary days go that high.

Standard deviation (SD) measures how much values usually jump around. If your app switches are 50, 52, 48, 51, 49, the SD is small (about 1.6). If they are 30, 70, 45, 60, 40, the SD is large (about 16).

Worked example

Say over her last 14 days, a user's app switches had a mean of **50** and an SD of **8**. Today she switched **82** times.

$z = (82 - 50) \div 8 = 32 \div 8 = \mathbf{+4.0}$

That is far outside her normal. Another user with a mean of 90 and SD 20 who also switched 82 times gets $z = (82 - 90) \div 20 = \mathbf{-0.4}$, a slightly quiet day *for him*. Same raw number, opposite meaning. This is why personal z-scores work.

The code

```
WINDOW, MIN_DAYS = 14, 7
g = df.groupby("user_id")
for c in BASE_COLS:                                          # 9 signals
    prev = g[c].shift(1)                                     # (1) yesterday and before
only
    mu = prev.groupby(df["user_id"]).transform(
            lambda s: s.rolling(WINDOW, min_periods=MIN_DAYS).mean())   # (2) mean of last
14
    sd = prev.groupby(df["user_id"]).transform(
            lambda s: s.rolling(WINDOW, min_periods=MIN_DAYS).std())    # (3) SD of last 14
    sd = sd.clip(lower=0.10 * mu.abs() + 1e-6)               # (4) safety floor
    df[f"z_{c}"] = (df[c] - mu) / sd                         # (5) the z-score
```

Line by line:

1. `shift(1)` moves every value down one day *within each user*. So the baseline for 15 March uses days up to 14 March, never 15 March itself.
2. `rolling(14).mean()` averages the last 14 of those values.
3. Same for the standard deviation.
4. **The floor.** If someone had a very steady fortnight, the SD could be tiny, and dividing by a tiny number explodes the z-score. So the SD is never allowed below 10% of the mean.
5. The z-score formula.

`min_periods=7` means "start once there are at least 7 past days". The first 7 days of each user have no baseline yet, so they are dropped (the **warm-up**). 4,800 − 40×7 = **4,520** rows remain for modelling.

> **The most important mistake to avoid: data leakage.** If today's value were included in today's baseline, the model would be secretly peeking at the answer, and its scores would look better than they really are. `shift(1)` prevents this. Leakage means *information the model would not have at prediction time sneaks into training*.

The deviation score

One combined number summarising "how distracted does today look compared with my normal":

```
df["deviation_score"] = (df["z_social_media_min"] + df["z_gaming_min"] +
df["z_app_switches"]
                         + df["z_night_usage_min"] - df["z_productive_min"]) / 5
```

Productive time is **subtracted**, because *less* productive time means *more* distracted. This score is the input to our simplest model (Chapter 13).

Q: How did you prevent leakage in your baseline? The rolling mean and SD use `shift(1)`, so a day's baseline only uses previous days. And the test month comes after the training months, so the model never sees the future.

CHAPTER 9

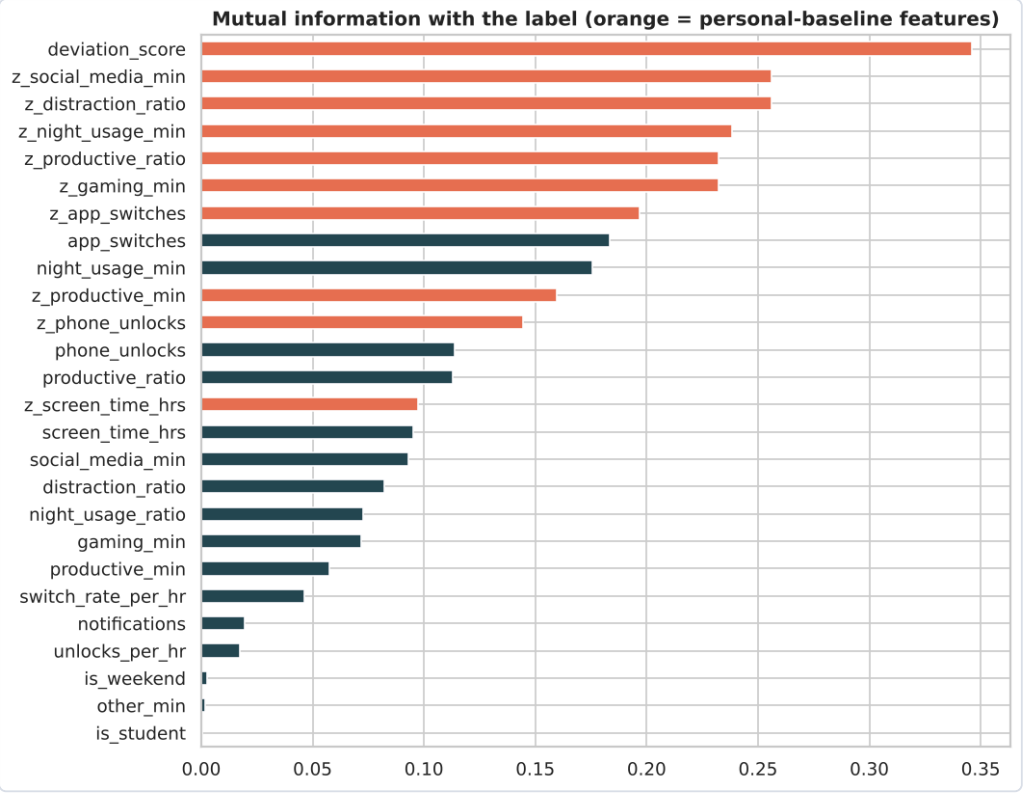
Choosing features and combining tables

9.1 Feature selection with mutual information

We now have 26 candidate columns (11 raw, 5 ratios, 10 personal-baseline). Not all are useful. **Feature selection** keeps the helpful ones.

> **Mutual information (MI)** measures how much knowing a feature reduces your uncertainty about the label. 0 means "tells you nothing". Higher means "tells you a lot". Unlike correlation, it also catches curved (non-linear) relationships.

```
from sklearn.feature_selection import mutual_info_classif
mi = pd.Series(mutual_info_classif(feat[CANDIDATES], feat["label_code"], random_state=42),
               index=CANDIDATES).sort_values(ascending=False)
```



Mutual information of each candidate feature. Orange bars are personal-baseline features: they dominate the top of the list.

The top six are all personal-baseline features (`deviation_score` 0.35, `z_social_media_min` 0.26 ...). The weakest are `is_weekend`, `other_min` and `is_student` (almost 0).

Two rules then remove features:

1. **Near-useless**: MI below 0.005 → dropped (`is_weekend`, `is_student`, `other_min`).
2. **Duplicates**: if two features are more than 90% correlated, they carry the same information, so we keep only the one with higher MI. (In our data no pair crossed 0.9, so this rule removed

nothing, but it is good practice to check.)

23 features remain.

9.2 Combining tables (data integration)

We build a second, smaller table with one row per user (their average screen time, average switches, % of distracted days) and join it to the daily table with `merge`, like VLOOKUP in Excel:

```
profile = df.groupby("user_id").agg(avg_screen_hrs=("screen_time_hrs", "mean"),
                                    avg_switches=("app_switches", "mean"))
feat = feat.merge(profile, on="user_id", how="left")
```

`how="left"` keeps every day even if something were missing in the profile. We also create **subsets** (students, professionals, weekdays, weekends) to compare label mixes, and a **pivot table** (label share per day of week) for the trend chart. These satisfy the "data integration" and "aggregate / reshape" deliverables.

✓ **Check yourself.** (1) Compute the z-score: mean 120, SD 30, today 60. Is that a distracted sign for *productive* minutes? (2) Why are the first 7 days of each user dropped? (3) What does MI = 0 mean?

Answer to (1): $z = (60 - 120) / 30 = -2$. For productive time, a big negative value is a strong distraction sign.

CHAPTER 10

Exploring the data (EDA)

Exploratory Data Analysis means looking at the data with statistics and charts before modelling, to understand it and to decide which models make sense. There are three levels:

- **Univariate**: one variable at a time (its distribution).
- **Bivariate**: two variables together (e.g., a feature versus the label).
- **Multivariate**: many at once (correlation heatmap, PCA).

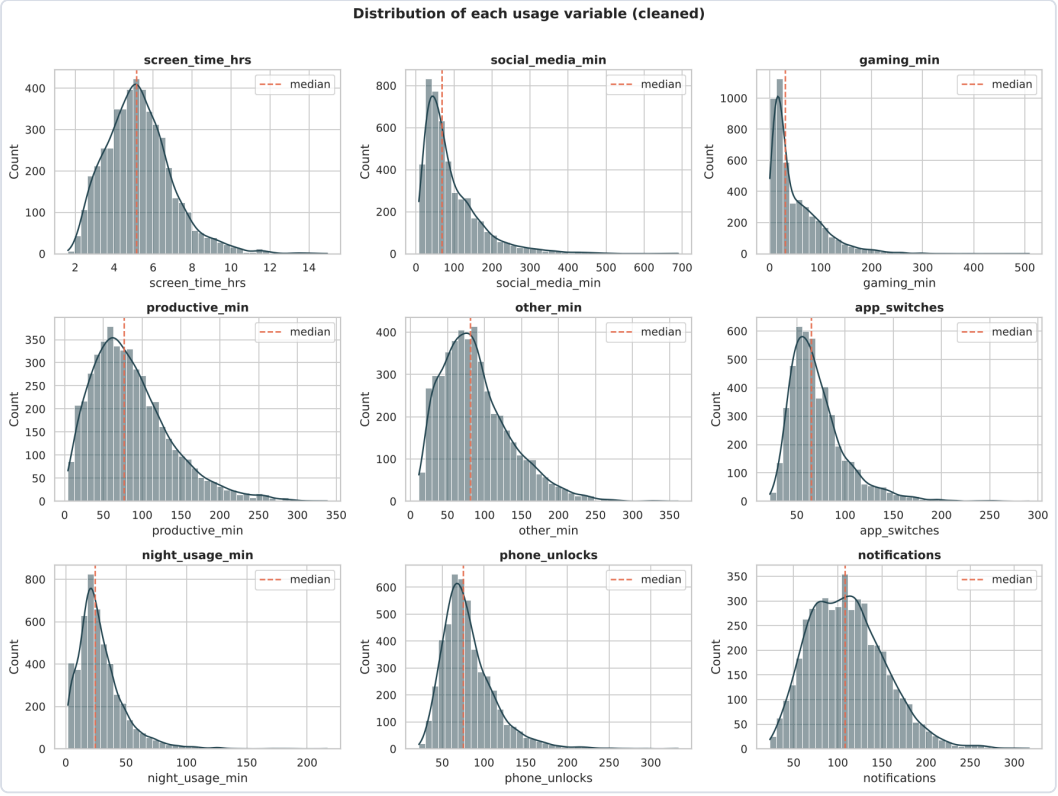
10.1 Descriptive statistics

`df.describe()` gives count, mean, std, min, quartiles and max. We also add:

- **Skewness**: whether the tail is longer on the right (positive) or left (negative). Our usage columns are right-skewed: many normal days, a few very heavy days.
- **Kurtosis**: how heavy the tails are (how often extreme values happen).

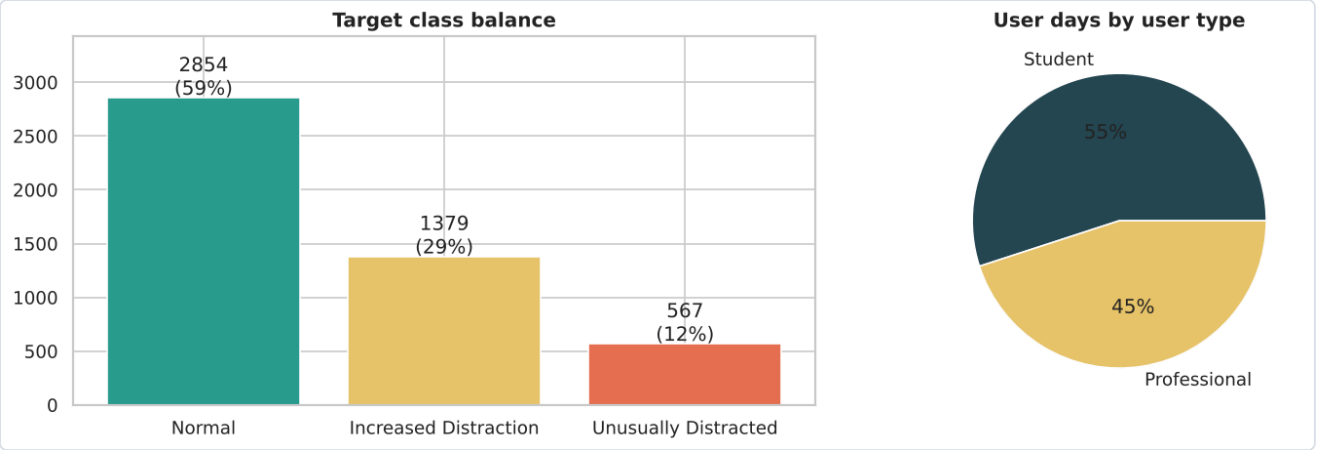
10.2 Univariate: histograms

A **histogram** splits values into bins and counts how many fall in each. It shows the *shape* of a variable.



Histograms of each usage variable. Most are right-skewed with long tails to the right; the dashed line is the median.

How to read it: the peak is where most days are; a long tail to the right means occasional very high days. Because the data is skewed, we used the median for imputation and chose models that handle skew.

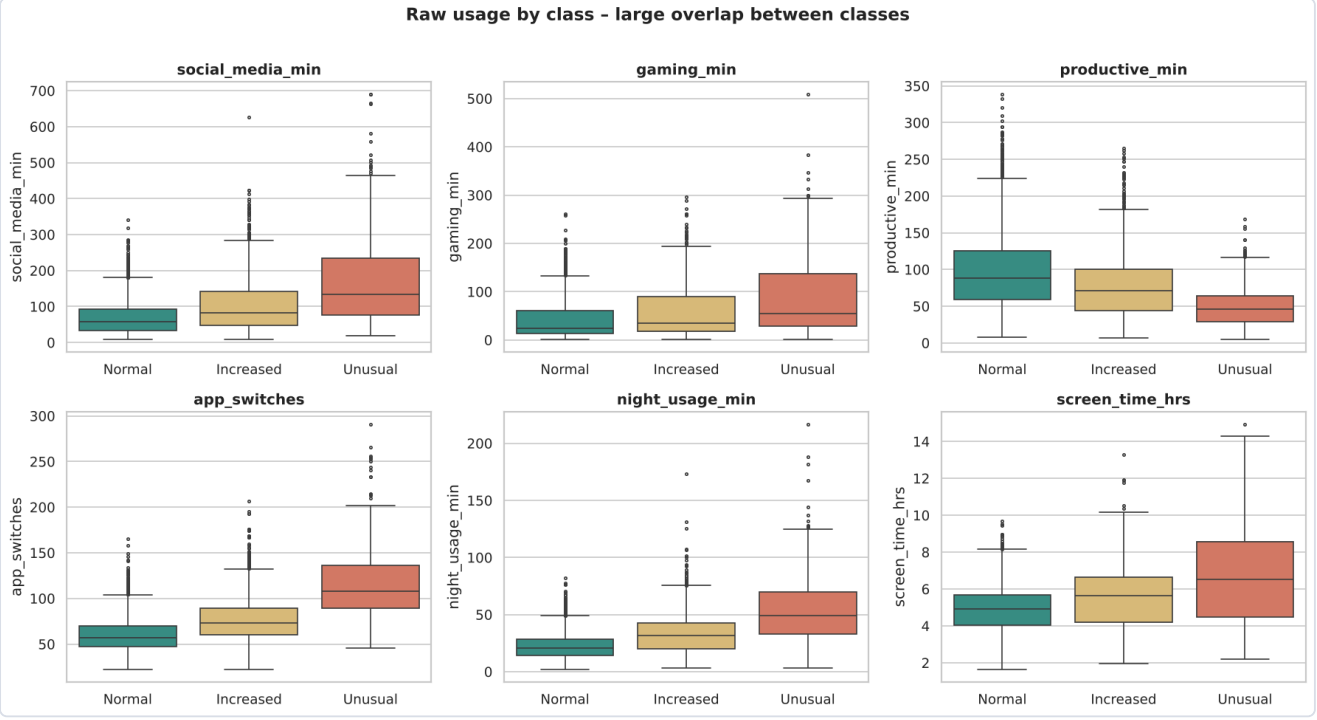


Class balance: about 60% Normal, 29% Increased, 12% Unusual.

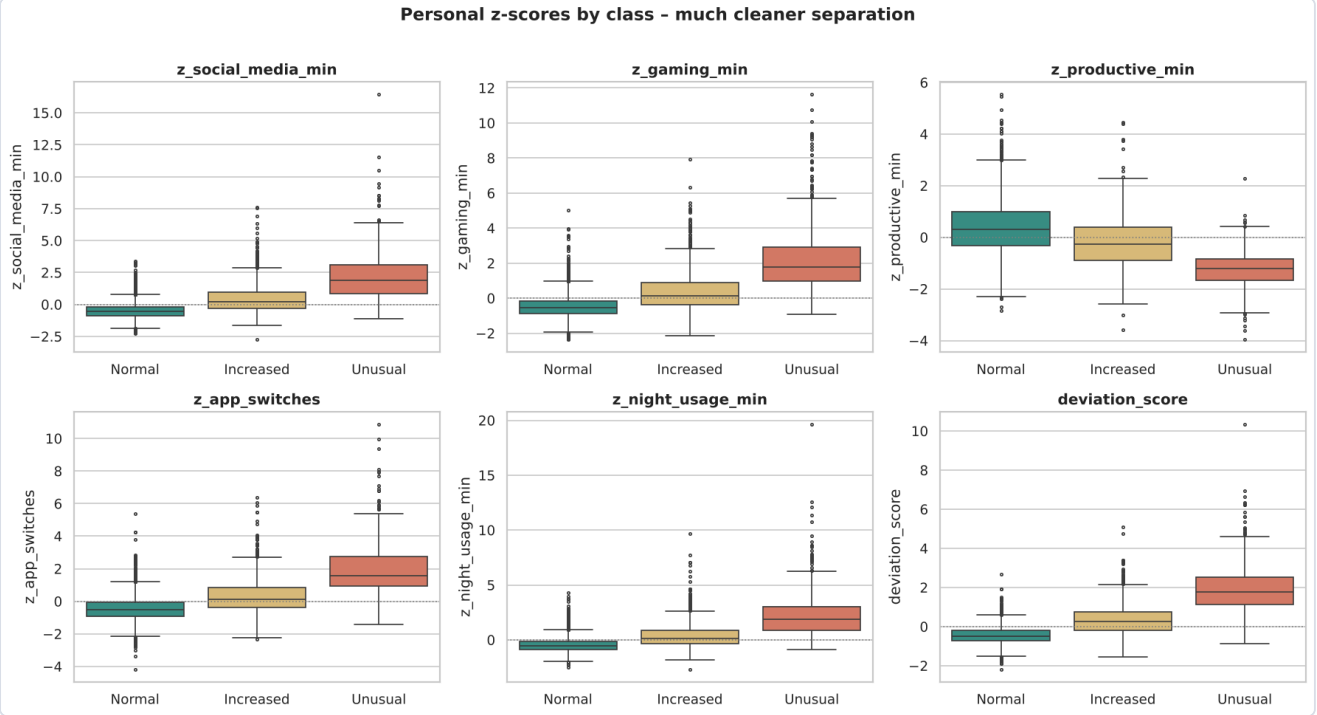
Class imbalance. When one class is much bigger, a lazy model that always says "Normal" would already be 60% accurate while being useless. That is why we judge models with **macro-F1** (Chapter 12), which treats all three classes as equally important.

10.3 Bivariate: box plots

A **box plot** shows a distribution compactly: the box covers the middle 50% of values (Q1 to Q3), the line inside is the median, whiskers reach out to typical extremes, and dots are outliers.



Raw usage by class: the boxes overlap a lot. Raw numbers alone separate the classes poorly.



The same signals as personal z-scores: the boxes move apart clearly. This is the main EDA finding.

How to read the pair: in the raw chart, the Normal box for app switches overlaps heavily with the Unusual box, because a heavy user's normal day looks like a light user's distracted day. In the z-score chart, the boxes separate cleanly. **This chart justifies the whole personal-baseline approach.**

10.4 A statistical test: Kruskal-Wallis

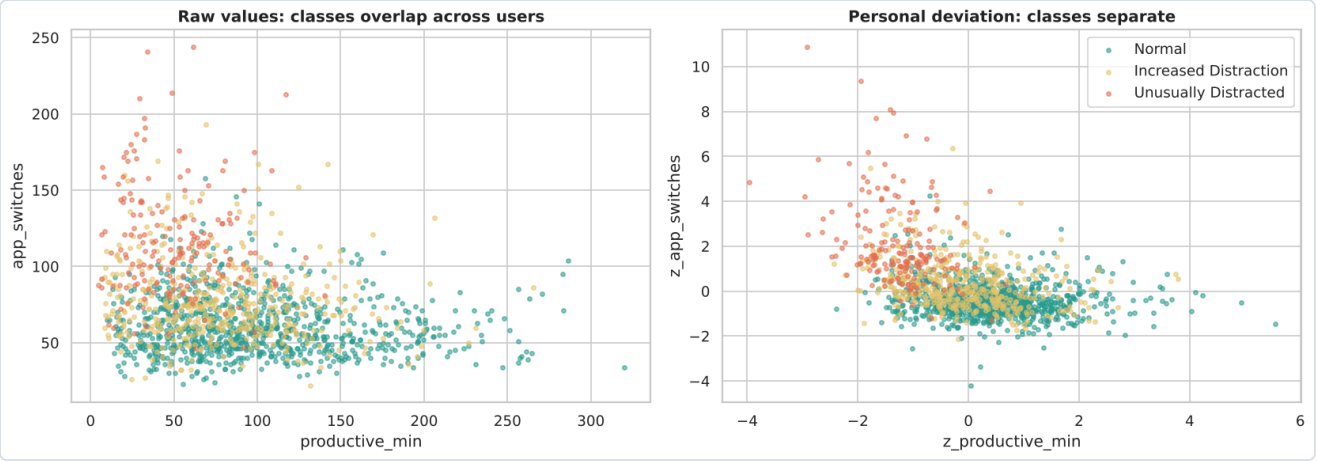
A chart can fool your eye, so we also test it.

Kruskal-Wallis test. Asks: "Do these groups (Normal, Increased, Unusual) have the same typical value?" It works on ranks, so it does not need the data to be bell-shaped (it is *non-parametric*). A tiny p-value (like 1e-200) means "the groups clearly differ". The **effect size ε^2** says *how much* they differ: closer to 1 is bigger.

```
from scipy import stats
h, p = stats.kruskal(*[feat.loc[feat.label == k, "z_app_switches"] for k in CLASSES])
```

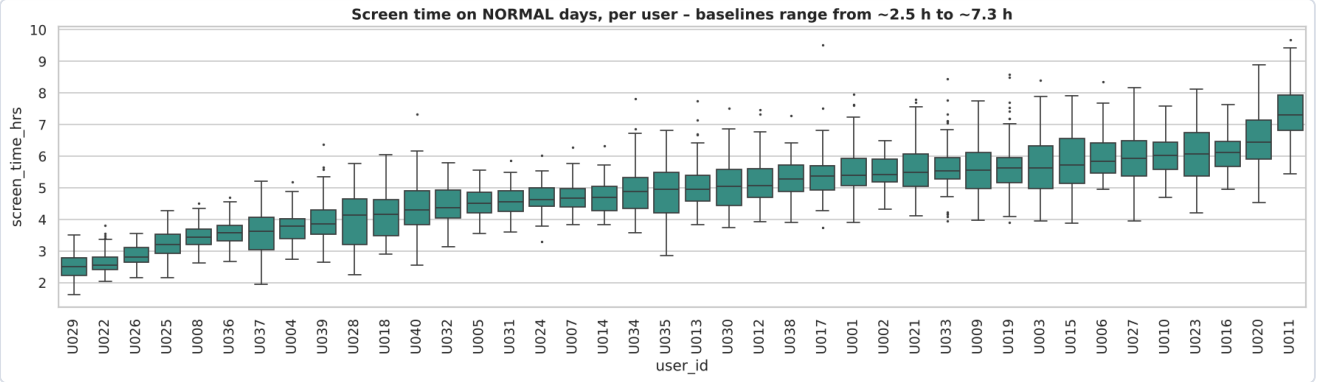
The `*` unpacks the list of three groups into three arguments. The z-score features have far larger effect sizes than the raw features.

10.5 Scatter plots: raw versus personal



Left: raw productive minutes vs app switches, colours overlap. Right: the same as personal z-scores, the colours form separate regions.

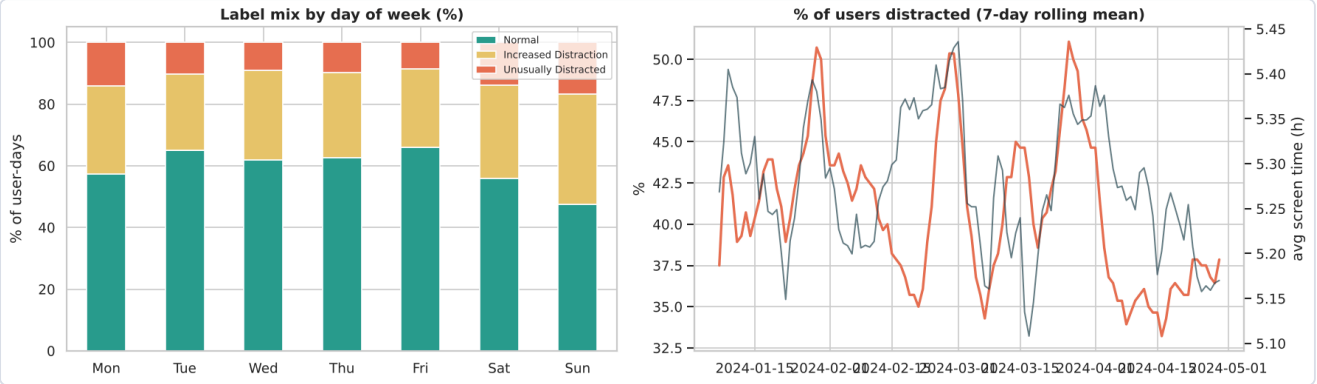
10.6 Why one threshold for everybody fails



Median screen time on normal days for each of the 40 users, sorted. Normal levels range from about 2.5 h to 7.3 h.

If the rule were "more than 6 hours = distracted", the heaviest users would be flagged every day and the lightest would never be flagged.

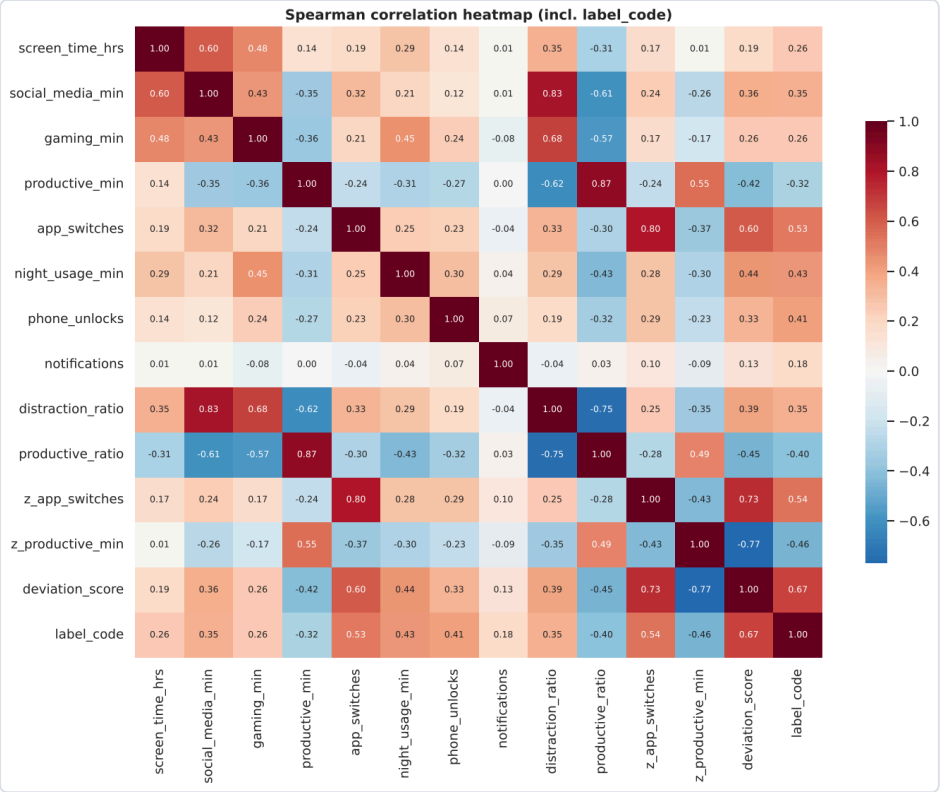
10.7 Trends



Left: label mix by day of week (weekends have more distracted days). Right: share of distracted users over time.

10.8 Multivariate: correlation heatmap

Correlation (from −1 to +1) measures how two variables move together: +1 always up together, −1 one up when the other goes down, 0 no straight-line link. We use **Spearman** correlation, which works on ranks and is safer for skewed data.

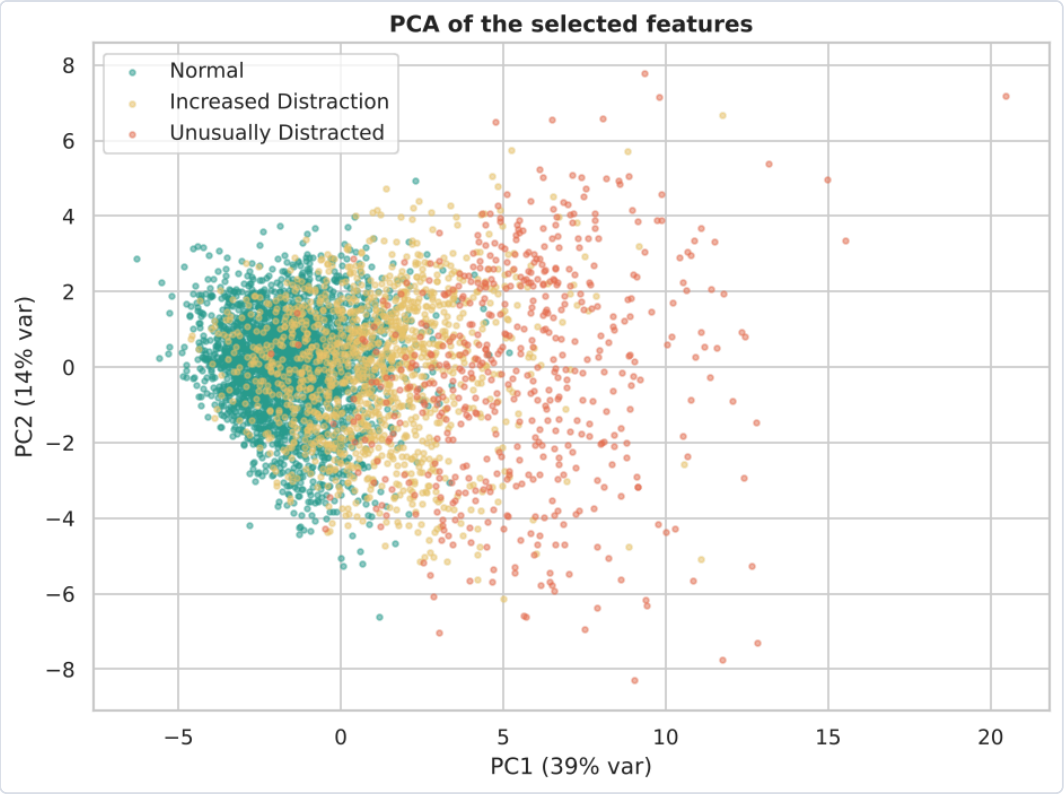


Spearman correlation heatmap. Red = move together, blue = move in opposite directions. The last row shows each feature's link with the label.

How to read it: look at the `label_code` row. Strong red cells are features that rise with distraction (`deviation_score`, switches); blue cells fall with it (productive ratio).

10.9 PCA: squeezing many features into a 2-D picture

PCA (Principal Component Analysis) finds the two directions in which the data spreads out most and draws every day in those two directions. It is a way to *see* 23 dimensions on flat paper.



PCA of the selected features. Classes are partly separated but overlap at the boundaries.

10.10 What EDA told us (and how it chooses the models)

Finding	Consequence for modelling
Classes are imbalanced (60 / 29 / 12%)	Use macro-F1, try `class_weight="balanced"`
Raw values overlap across people	Use personal z-score features
z-scores separate the classes	Even simple models may work well
Weekend effect already shows in the usage	`is_weekend` adds little on its own
Some interactions and curved boundaries	Compare a linear model with tree models
People rarely label their days	Also test unsupervised methods
Skewed, heavy tails	Robust median, tree models, scaling inside pipelines

Q: How did EDA influence your model selection? EDA showed raw values overlap but personal z-scores separate the classes, so I built z-score features; imbalance led me to macro-F1 and balanced class weights; partly non-linear boundaries made me compare linear Logistic Regression against tree models; and because labels are rare in real life I added Isolation Forest and K-Means.

✓ **Check yourself.** (1) In a box plot, what does the box show? (2) What does a p-value of 1e-100 in Kruskal-Wallis mean? (3) Why Spearman and not Pearson correlation here?

CHAPTER 11

How machine learning works (the core ideas)

11.1 What a model is

A **model** is a formula with adjustable knobs. **Training** (or *fitting*) means turning the knobs until the formula's answers match the known labels as closely as possible. Once trained, the formula can give answers for new days it has never seen.

In scikit-learn every model works the same way, which makes it easy to learn:

```
model = LogisticRegression()     # 1. create the model
model.fit(X_train, y_train)      # 2. learn from training data
y_pred = model.predict(X_test)   # 3. predict labels for new data
proba  = model.predict_proba(X_test)   # 4. probability of each class
```

`X` is the table of features (one row per day). `y` is the column of correct labels.

11.2 Train and test: the fair exam

If you test a student with the exact questions they practised, a high score proves nothing. Models are the same. So we split the data:

- **Training set**: the model learns from it.
- **Test set**: kept hidden until the very end, used once to measure real performance.

We split **by time**: train on 8 January – 31 March (3,360 days), test on 1 – 29 April (1,160 days).

```
train = feat[feat["date"] <  "2024-04-01"]
test  = feat[feat["date"] >= "2024-04-01"]
```

Why split by time instead of randomly? In real use the model always predicts *future* days. A time split copies that. A random split would let the model learn from 15 April and then be tested on 14 April, which is a mild form of cheating.

11.3 Overfitting and underfitting

- **Overfitting**: the model memorises the training data, including its noise, and does badly on new data. Like a student who memorised answers without understanding. A single deep Decision Tree is the classic example.
- **Underfitting**: the model is too simple to capture the pattern. Like a student who only learnt one rule.

We want the middle: simple enough to generalise, flexible enough to capture the pattern. Tuning (Chapter 14) finds that middle.

11.4 Cross-validation

The test set may only be used once, at the end. So how do we compare settings during development? **Cross-validation (CV)**: split the *training* data into 5 parts (folds), train on 4 and check on the 5th, rotate 5 times, average the 5 scores.

```
Fold 1:  [check] [train] [train] [train] [train]
Fold 2:  [train] [check] [train] [train] [train]
 ...
Fold 5:  [train] [train] [train] [train] [check]
```

GroupKFold: the extra-fair version

Our rows are days, and days from the same person are similar to each other. If a person's days are in both the train and check parts, the model "recognises" the person and scores too high. **GroupKFold** keeps each person entirely in one fold, so every check is on **people the model has never seen**.

```python
from sklearn.model_selection import GroupKFold, cross_val_score
cv = GroupKFold(n_splits=5)
scores = cross_val_score(model, X_tr, y_tr, groups=grp_tr, cv=cv, scoring="f1_macro")
print(scores.mean(), scores.std())
```

`groups=grp_tr` passes the user id of each row. The **standard deviation** of the 5 scores tells you how stable the model is: small = reliable.

11.5 Scaling and pipelines

Some models (Logistic Regression, K-Means) are confused when features are on very different scales (app switches around 60, ratios around 0.4). **StandardScaler** rescales every feature to mean 0, SD 1 (a z-score of each column).

The scaler must learn its mean and SD **from training data only**, otherwise information from the test set leaks in. A **Pipeline** guarantees this:

```python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
model = Pipeline([("sc", StandardScaler()),
                  ("clf", LogisticRegression(max_iter=2000))])
model.fit(X_tr, y_tr)        # scaler fitted on X_tr only, then the model
```

Tree models (Decision Tree, Random Forest) only ask "is this value above or below a cut?", so scaling does not change them.

✓ **Check yourself.** (1) What is the difference between the test set and cross-validation? (2) Why GroupKFold instead of normal KFold? (3) Why must the scaler be inside the pipeline?

CHAPTER 12

Measuring a model: the metrics

Review 3 marks you on "appropriate use of evaluation metrics", so understand these well.

12.1 The confusion matrix

A table counting what the model said against what was true. Here is the real one for our final model on the 1,160 test days:

Actual ↓ / Predicted →	Normal	Increased	Unusual	Total
Normal	**682**	60	1	743
Increased	110	**172**	20	302
Unusual	0	27	**88**	115

The **diagonal** (bold) is correct. Everything else is a mistake. Notice: only 1 Normal day was called Unusual, and no Unusual day was called Normal. All mistakes are between *neighbouring* classes.

12.2 Accuracy

Share of all predictions that were correct:

Accuracy = (682 + 172 + 88) ÷ 1,160 = 942 ÷ 1,160 = **0.812**

Weakness: with imbalanced classes it flatters a model that is good only at the big class.

12.3 Precision and recall (for one class at a time)

Take the class **Unusual**:

- **Precision** = of the days the model *called* Unusual, how many really were? = 88 ÷ (1 + 20 + 88) = 88 ÷ 109 = **0.807**. *"When it raises the alarm, how often is it right?"*
- **Recall** = of the days that really *were* Unusual, how many did the model catch? = 88 ÷ 115 = **0.765**. *"How many of the bad days did it find?"*

> **Memory trick.** Precision is about the model's *column* (its predictions). Recall is about the true *row* (reality). A smoke alarm that beeps for toast has low precision. One that misses real fires has low recall.

12.4 F1 score

One number balancing precision and recall (their *harmonic mean*):

F1 = 2 × P × R ÷ (P + R) = 2 × 0.807 × 0.765 ÷ (0.807 + 0.765) = **0.786**

It is high only when both are high.

12.5 Macro averaging

Compute F1 for each class, then take the plain average. Each class counts equally, however small:

Class	Precision	Recall	F1
Normal	0.861	0.918	0.889
Increased	0.664	0.570	0.613
Unusual	0.807	0.765	0.786
Macro average	0.778	0.751	**0.763**

Macro-F1 is our main metric, because it cannot be fooled by the big Normal class.

12.6 ROC curve and ROC-AUC

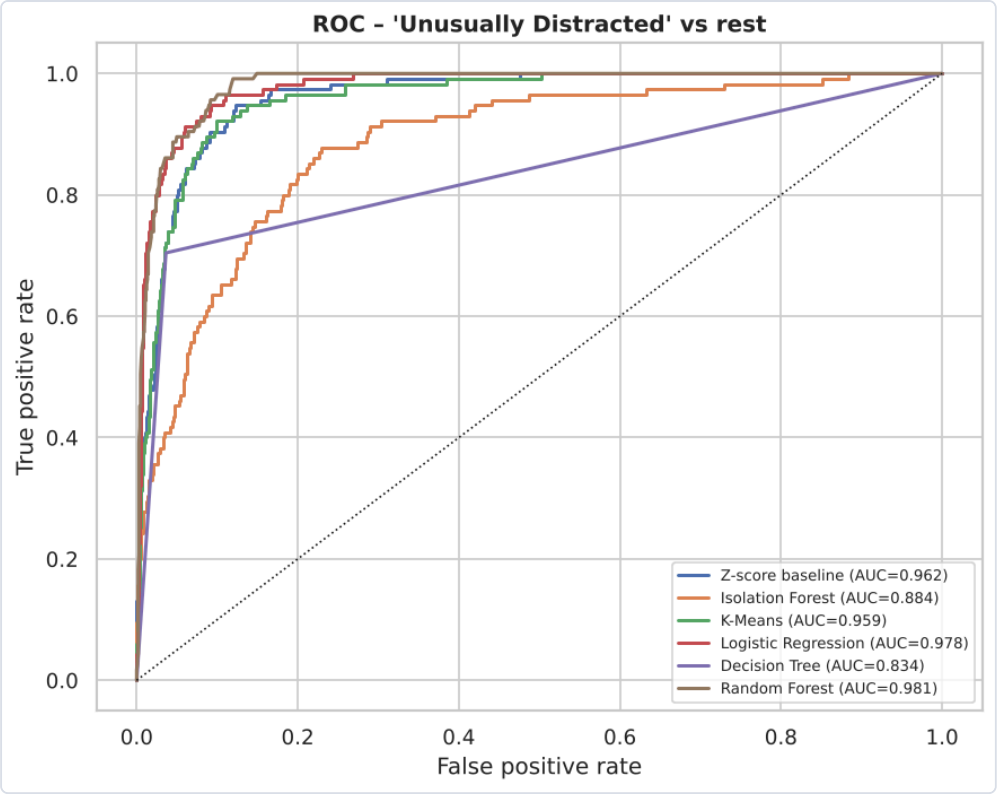
Models also give a **probability** for each class. To decide "Unusual or not", you choose a cut-off (say 0.5). Every cut-off gives a different trade-off:

- **True positive rate** (= recall): share of real Unusual days caught.
- **False positive rate**: share of non-Unusual days wrongly flagged.

The **ROC curve** plots these for every possible cut-off. **AUC** (area under the curve) is one number from 0.5 (coin toss) to 1.0 (perfect).

> **AUC in plain words.** Pick one random Unusual day and one random other day. AUC is the chance the model gives the Unusual day the higher score. Our final model: 0.898, so about 90% of the time.

For three classes we compute "this class vs the rest" for each and average (**one-vs-rest, OvR**).



ROC curves for "Unusually Distracted vs the rest", all six models. Curves closer to the top-left corner are better.

12.7 Clustering metrics (for K-Means)

Clustering has no labels to compare against, so it is judged by how tidy the groups are:

Metric	Measures	Better when
Silhouette (−1 to 1)	Is each point closer to its own group than to the next group?	Higher
Davies-Bouldin	Average similarity between each group and its most similar group	Lower
Calinski-Harabasz	Spread between groups ÷ spread inside groups	Higher

12.8 The code

```
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, confusion_matrix)
accuracy_score(y_te, y_pred)
precision_score(y_te, y_pred, average="macro")
recall_score(y_te, y_pred, average="macro")
f1_score(y_te, y_pred, average="macro")
f1_score(y_te, y_pred, labels=[2], average="macro")         # F1 for class 2 (Unusual) only
roc_auc_score(y_te, proba, multi_class="ovr", average="macro")
confusion_matrix(y_te, y_pred)
```

The notebook wraps these in one helper, `evaluate(name, y_pred, proba, cv_f1)`, so all six models are measured in exactly the same way and stored in one `results` dictionary.

Q: Why macro-F1 and not accuracy? The classes are imbalanced (60% Normal). Accuracy rewards a model that is only good at Normal. Macro-F1 averages the F1 of each class equally, so the model must also do well on the small but important Unusual class.

✓ **Check yourself.** Using the confusion matrix, compute precision and recall for *Increased*. (Answer: 172 ÷ 259 = 0.664 and 172 ÷ 302 = 0.570.)

CHAPTER 13

The six models, one by one

For each model: the idea in plain words, how it decides, the code, how it did, and its strengths and weaknesses. All results are on the April test set.

13.1 Model 1: Z-score rule (statistical baseline)

Idea. No learning at all. Use the deviation score from Chapter 8 with the textbook rule: 1 SD above normal = Increased, 2 SD = Unusual.

```
T1, T2 = 1.0, 2.0
score = test["deviation_score"].values
pred = np.where(score >= T2, 2, np.where(score >= T1, 1, 0))
```

The nested `np.where` reads: "if score ≥ 2 → 2, else if score ≥ 1 → 1, else 0".

Why include it? It is the **baseline**: the simplest sensible method. Every ML model must beat it, or the ML is pointless.

Result: accuracy 0.716, macro-F1 **0.540**. With thresholds tuned on training data (0.4 and 1.4 SD): macro-F1 0.650.

Weakness: it weights every signal equally and uses one fixed rule for everyone.

13.2 Model 2: Isolation Forest (anomaly detection)

Idea. Unusual points are easy to separate from the crowd. The algorithm builds many random trees that keep cutting the data at random places. A strange day gets isolated after only a few cuts; an ordinary day needs many. **Fewer cuts = more anomalous.**

```
from sklearn.ensemble import IsolationForest
iso = Pipeline([("sc", StandardScaler()),
                ("iso", IsolationForest(n_estimators=300, random_state=42))])
iso.fit(train[Z_ONLY])                    # NO labels used!
anomaly = -iso.score_samples(test[Z_ONLY])  # higher = stranger
```

It gives a strangeness score, not a label. We turn it into three classes using the training class shares: the strangest ~12% → Unusual, the next ~29% → Increased.

Result: macro-F1 **0.518**, the lowest.

Why it struggles: it detects *unusual*, not *distracted*. An unusually **productive** day is also strange, so it gets flagged too. It does not know the direction of the change.

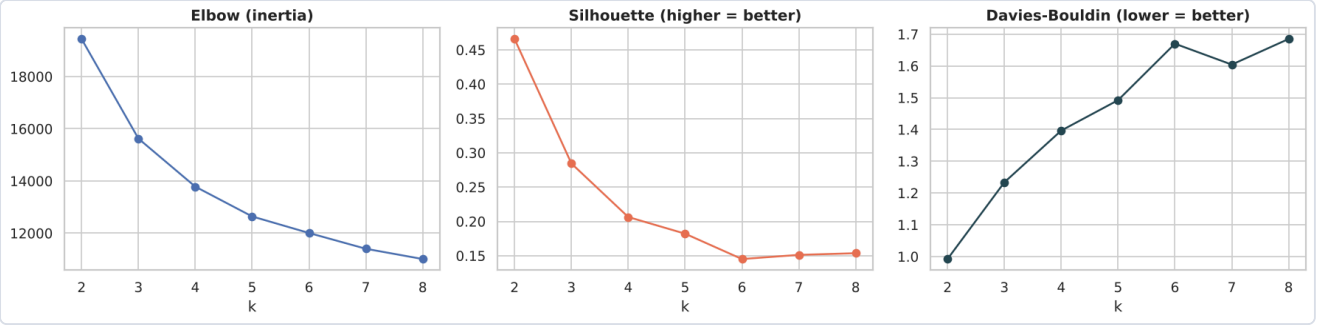
13.3 Model 3: K-Means clustering

Idea. Put the days into k groups so that days in the same group are close together.

1. Place k centre points at random.

2. Assign every day to its nearest centre.
3. Move each centre to the middle of its group.
4. Repeat 2–3 until nothing changes.

Choosing k. Try k = 2 to 8 and look at the metrics:



Elbow (inertia), silhouette and Davies-Bouldin for k = 2 to 8.

k = 2 has the best silhouette (0.47) because it splits "normal" from "not normal". We use **k = 3** because the problem has three states and the elbow still bends there. On the test days, k = 3 gives silhouette 0.283, Davies-Bouldin 1.247, Calinski-Harabasz 612.

To compare with the other models, each cluster is named after the most common label of its training days (*cluster-then-label*).

```
km = KMeans(n_clusters=3, n_init=20, random_state=42).fit(Xk_tr)
cluster_to_label = pd.crosstab(km.labels_, y_tr).idxmax(axis=1).to_dict()
pred = np.array([cluster_to_label[c] for c in km.predict(Xk_te)])
```

Result: macro-F1 **0.674**, better than the rule, but below the supervised models.

Weakness: K-Means assumes round, similar-sized groups; our classes are uneven (60/29/12) and ordered.

13.4 Model 4: Logistic Regression (the winner)

Idea. Give each feature a **weight**. Multiply each feature by its weight and add them up to get a score for each class. Turn the three scores into probabilities that add to 1 with the **softmax** function.

score(Unusual) = b + w_1 × $feature_1$ + w_2 × $feature_2$ + … + w_{23} × $feature_{23}$

probability(class) = e^score(class) ÷ (e^score(Normal) + e^score(Increased) + e^score(Unusual))

Training finds the weights that make the correct labels as probable as possible.

Reading a weight. After scaling, a weight of +0.54 for `app_switches` in the Unusual score means: "one SD more app switching than average raises the log-odds of an Unusual day by 0.54." Positive = pushes towards that class, negative = pushes away.

Real weights from our model for four of the strongest features:

Feature	Normal	Increased	Unusual
app_switches	−0.52	−0.02	+0.54
notifications	−0.41	−0.04	+0.45
z_gaming_min	−0.38	+0.04	+0.34
phone_unlocks	−0.38	+0.05	+0.33

The C setting (regularisation). C controls how big the weights may grow. Small C = weights kept small = simpler, steadier model. Our tuned value is **C = 0.03**, a strong pull towards simplicity.

```
model = Pipeline([("sc", StandardScaler()),
                  ("clf", LogisticRegression(C=0.03, max_iter=2000))])
model.fit(X_tr, y_tr)
```

Result (tuned): accuracy **0.812**, macro-F1 **0.763**, Unusual F1 0.786, ROC-AUC 0.898. Best on 5 of 7 metrics.

Strengths: stable, gives real probabilities, every weight can be explained, tiny to deploy. **Weakness:** boundaries are straight lines (linear), so it cannot model complex interactions by itself.

13.5 Model 5: Decision Tree

Idea. A flowchart of yes/no questions learnt from the data. A simplified example of the shape (not the exact learnt tree):

```
Is deviation_score ≤ 0.6?
├── yes → Is app_switches ≤ 95?
│         ├── yes → Normal
│         └── no  → Increased
└── no  → Is z_productive_min ≤ −1.3?
          ├── yes → Unusual
          └── no  → Increased
```

At each step it picks the question that best separates the classes, measured by **Gini impurity** (how mixed the labels are in a group: 0 = all the same label).

Settings that control overfitting: `max_depth` (how many questions deep) and `min_samples_leaf` (smallest allowed group at the end).

Result: default (unlimited depth) macro-F1 **0.662** → it overfits. Tuned (depth 5, at least 40 days per leaf) → **0.703**.

Strength: easy to read. **Weakness:** one tree is unstable; small data changes give a different tree (high *variance*).

13.6 Model 6: Random Forest

Idea. Grow **hundreds of different trees** and let them vote. Each tree sees a random sample of the days and, at each question, only a random subset of the features. Their mistakes are different, so averaging cancels many of them out. This is called an **ensemble** and the sampling trick is called **bagging**.

```
rf = RandomForestClassifier(n_estimators=300, max_depth=10, min_samples_leaf=3,
                            max_features=0.5, class_weight="balanced", random_state=42)
```

Result: tuned macro-F1 **0.750**, accuracy 0.793. Very close to Logistic Regression, and the best cross-validation score (0.763).

Strength: handles curves, interactions and skew. **Weakness:** harder to explain, bigger to deploy, and its scores vary more between groups of people.

13.7 class_weight="balanced"

With imbalanced classes, a model can ignore the small class. `class_weight="balanced"` makes mistakes on rare classes cost more during training. It is one of the settings the grid search was allowed to try.

Q: Explain Random Forest in one sentence. Many decision trees, each trained on a random sample of rows and features, vote on the answer, which reduces the overfitting of a single tree.

✓ **Check yourself.** (1) Which models use no labels? (2) Why does Isolation Forest flag very productive days? (3) What does a smaller C do in Logistic Regression?

CHAPTER 14

Tuning the models

14.1 Hyperparameters vs parameters

- **Parameters** are learnt during training (the weights of Logistic Regression, the questions of a tree).
- **Hyperparameters** are settings *you* choose before training (C, max_depth, number of trees).

Tuning means searching for the hyperparameters that give the best cross-validated score.

14.2 Grid Search

Grid search tries **every combination** in a list you give it, scores each with cross-validation, and keeps the best.

```
from sklearn.model_selection import GridSearchCV
grid = {"max_depth": [3, 4, 5, 6, 8, 10],
        "min_samples_leaf": [5, 10, 20, 40],
        "criterion": ["gini", "entropy"],
        "class_weight": [None, "balanced"]}
gs = GridSearchCV(DecisionTreeClassifier(random_state=42), grid,
                  scoring="f1_macro", cv=GroupKFold(5), n_jobs=-1)
gs.fit(X_tr, y_tr, groups=grp_tr)
gs.best_params_        # the winning settings
gs.best_score_         # their CV macro-F1
gs.best_estimator_     # the model, already retrained with those settings
```

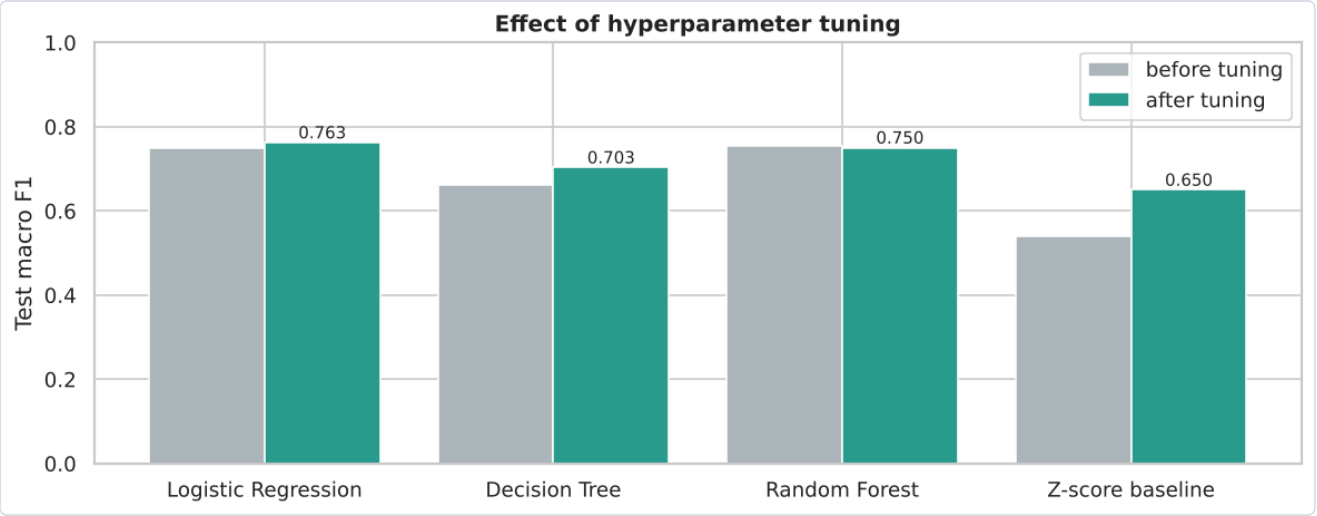
This grid has 6 × 4 × 2 × 2 = **96 combinations**, each trained 5 times (5 folds) = 480 fits. `n_jobs=-1` uses all CPU cores.

For a Pipeline, settings of a step are named `step__setting`, e.g. `"clf__C": [0.01, 0.03, 0.1, 0.3, 1, 3, 10]`.

Random Search is the alternative: try a random sample of combinations. It is faster for huge grids. Our grids are small, so Grid Search is fine.

14.3 Results before and after

Model	Best settings	Test macro-F1 before → after
Logistic Regression	C = 0.03, no class weight	0.749 → **0.763**
Decision Tree	depth 5, min leaf 40, gini	0.662 → **0.703**
Random Forest	300 trees, depth 10, max_features 0.5, balanced	0.754 → 0.750
Z-score rule	thresholds 0.4 / 1.4 SD	0.540 → **0.650**



Test macro-F1 before and after tuning.

Notice the Random Forest got very slightly *worse* on the test set even though its CV score improved (0.760 → 0.763). This is normal: the test month is one sample, and small ups and downs of ±0.01 are noise. Report it honestly.

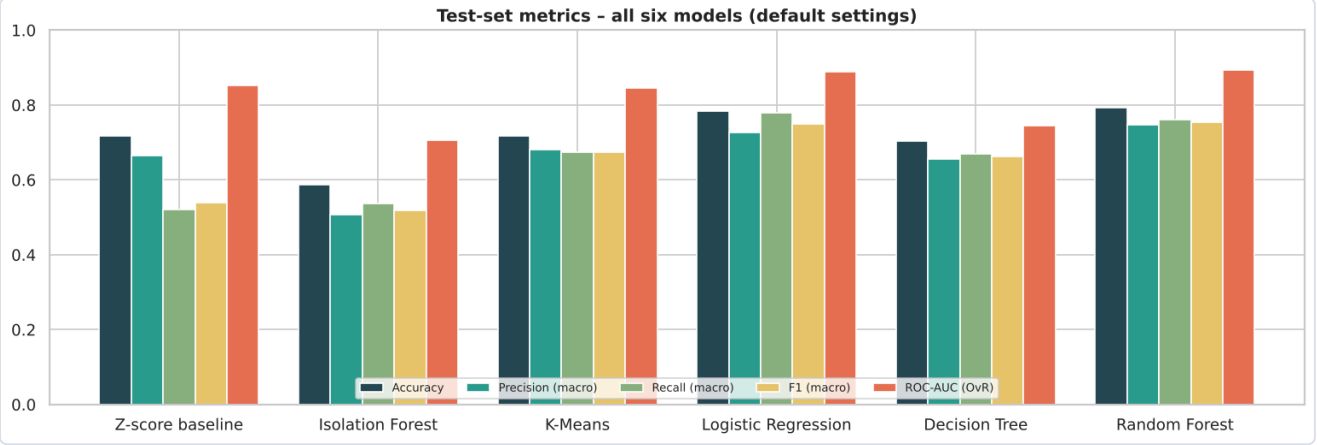
Q: How did you tune your models? GridSearchCV with GroupKFold 5-fold cross-validation by user, optimising macro-F1, on the training months only. The test month was used once at the end to compare before and after.

CHAPTER 15

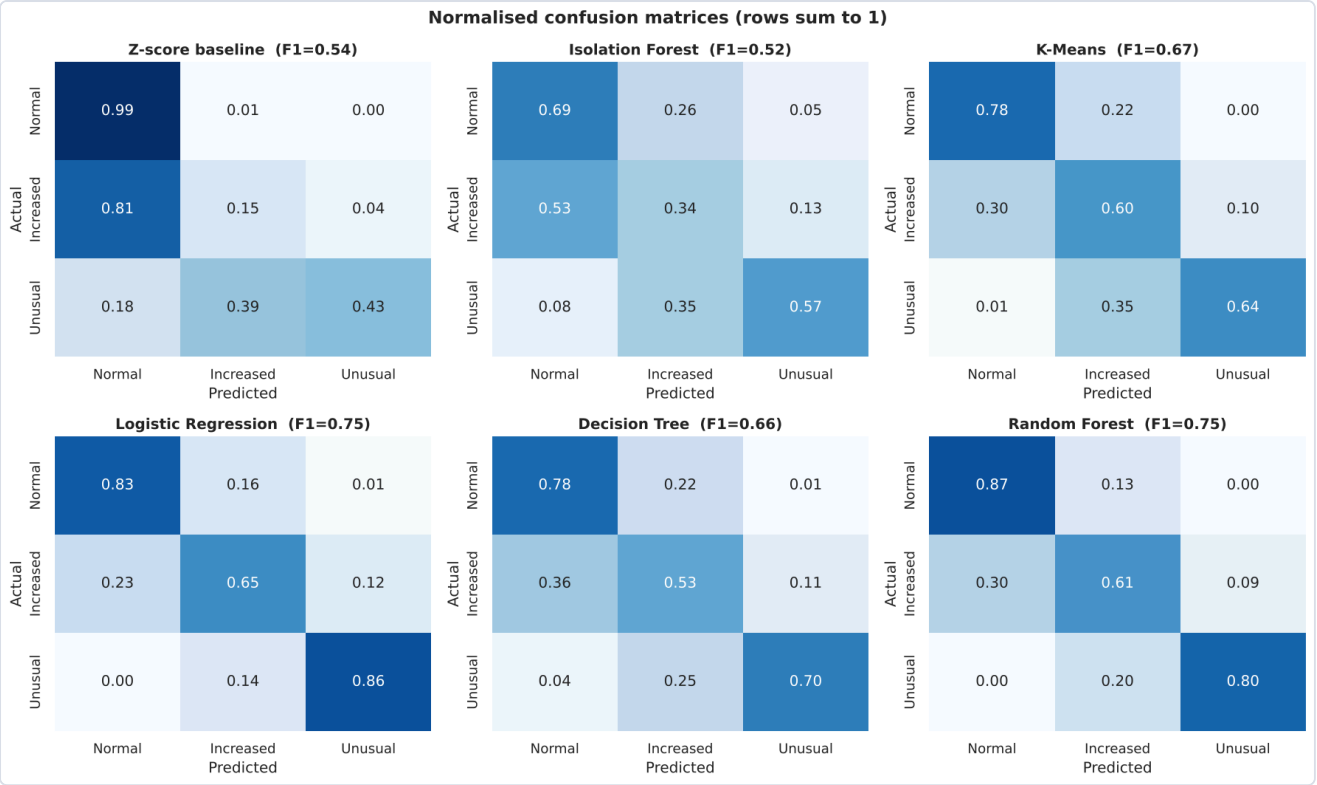
Comparing models and choosing the winner

15.1 The full comparison

Model	Accuracy	Macro-F1	F1 Unusual	ROC-AUC	CV F1
Z-score rule	0.716	0.540	0.554	0.853	–
Z-score rule (tuned)	0.754	0.650	0.667	0.855	–
Isolation Forest	0.588	0.518	0.510	0.705	–
K-Means	0.718	0.674	0.676	0.847	–
Decision Tree (tuned)	0.761	0.703	0.712	0.858	0.708
Random Forest (tuned)	0.793	0.750	0.762	0.893	**0.763**
Logistic Regression (tuned)	**0.812**	**0.763**	**0.786**	**0.898**	0.752



All six models (default settings) on the test month.



Confusion matrices, rows normalised so each row adds to 1. A strong diagonal is good.

15.2 The selection rule (decided in advance)

1. Among the tuned supervised models, count how many of the 7 metrics each one wins.
2. If two are close, check whether the gap is real or just noise.
3. If it is noise, prefer the **simpler** model.

Logistic Regression wins 5 of 7 metrics. Random Forest wins 2 (recall and CV-F1). So step 2 matters.

15.3 Is the difference real? Two tests

McNemar test. Look only at days where the two models disagree:

- LR right, RF wrong: **75 days**
- RF right, LR wrong: **53 days**

If the models were equally good, these two numbers would be about equal. The test asks how surprising 75 vs 53 is. **p = 0.063**. The usual cut-off is 0.05, so this is **not** statistically significant.

```
n01, n10 = 75, 53
p = stats.binomtest(min(n01, n10), n01 + n10, 0.5).pvalue
```

Bootstrap confidence interval. Resample the 1,160 test days with replacement 1,000 times, recompute the F1 difference each time, and look at the middle 95%:

```
for _ in range(1000):
    i = rng.integers(0, len(y_te), len(y_te))     # random days, repeats allowed
    diffs.append(f1(y_te[i], pred_lr[i]) - f1(y_te[i], pred_rf[i]))
np.percentile(diffs, [2.5, 97.5])
```

Result: difference **+0.012**, 95% interval **−0.015 to +0.042**. The interval includes 0, so again: a tie.

p-value in plain words. If there were truly no difference, how often would we see a gap this big just by luck? p = 0.063 means about 6% of the time, which is not rare enough to claim a real difference.

15.4 Why Logistic Regression wins the tie

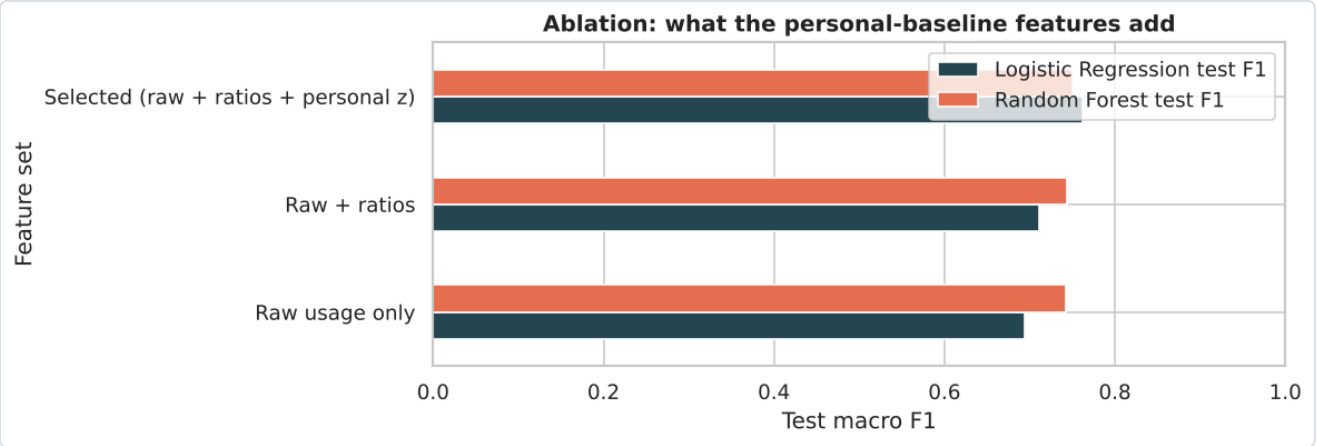
Reason	Evidence
Best on 5 of 7 metrics	Comparison table
More stable across unseen people	CV spread ±0.014 vs ±0.032 for RF
Explainable	Each weight has a meaning (Chapter 13)
Calibrated probabilities	Its probabilities are meaningful percentages
Tiny and fast	1.7 kB, runs inside a web page

This follows **Occam's razor**: when two explanations are equally good, prefer the simpler one.

15.5 Ablation: proving the personal baseline matters

An **ablation study** removes one ingredient to see how much it contributed. We retrained both top models on three feature sets:

Features	LR macro-F1	RF macro-F1
Raw usage only (11)	0.694	0.742
Raw + ratios (16)	0.711	0.743
Raw + ratios + personal z-scores (23)	**0.763**	0.750



Ablation chart.

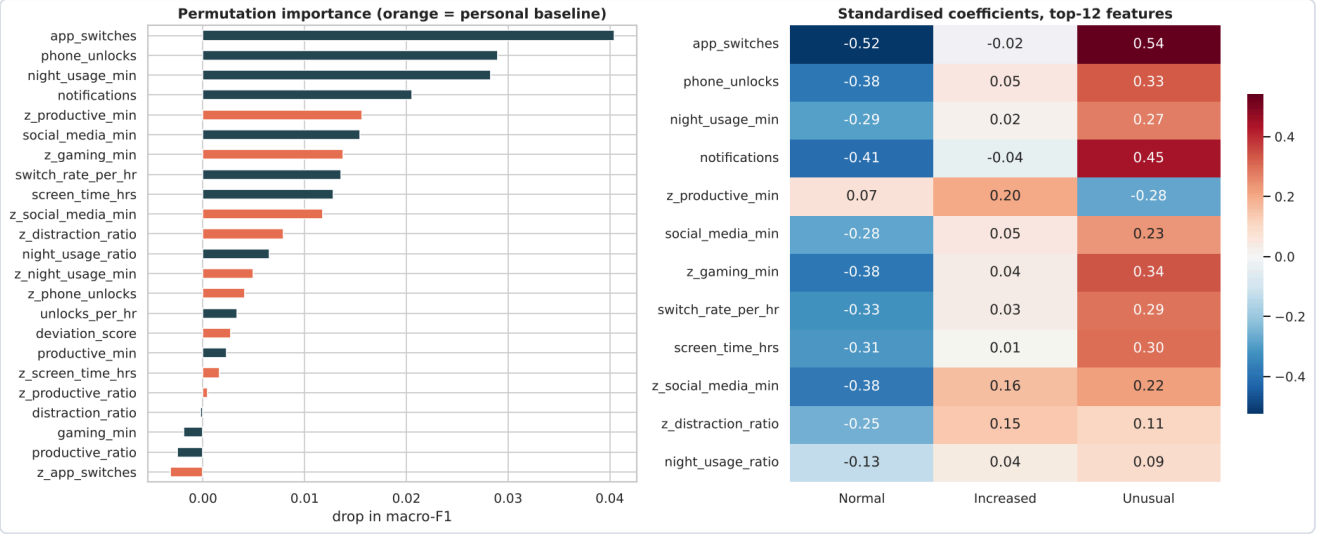
What this shows: the personal-baseline features lift Logistic Regression by **+0.069**. Once each day is expressed as "distance from my normal", the boundary between classes becomes nearly a straight line, which is exactly what a linear model handles well. The forest gains less because it was already approximating those comparisons with its many splits.

15.6 Which features matter most

Permutation importance: shuffle one feature's column (destroying its information) and measure how much macro-F1 drops. A big drop = an important feature.

```
from sklearn.inspection import permutation_importance
pi = permutation_importance(final_model, X_te, y_te, scoring="f1_macro",
                            n_repeats=15, random_state=42)
```

Top features: `app_switches`, `phone_unlocks`, `night_usage_min`, `notifications`, then `z_productive_min`, `social_media_min`, `z_gaming_min`.



Left: permutation importance. Right: Logistic Regression weights for the top 12 features.

Careful when reading importance. Raw and z-score versions of the same signal share information, so shuffling one at a time understates each. The ablation above is the clearer test of what the personal baseline adds.

15.7 Error analysis

- 218 of 1,160 test days are wrong (18.8%).
- 217 of the 218 errors are between **neighbouring** classes, mostly Increased mistaken for Normal.
- Only **1** Normal day was called Unusual (the worst kind of false alarm).

Neighbouring-class errors are expected: distraction is gradual, and about 7% of labels are noisy on purpose.

Q: Random Forest had a higher CV score. Why didn't you choose it? The gap between them is not statistically significant (McNemar p = 0.063, bootstrap CI includes 0). With equal accuracy I chose the simpler model: Logistic Regression is more stable across unseen people, interpretable, gives calibrated probabilities, and runs in the browser.

✓ **Check yourself.** (1) What does a 95% confidence interval that includes 0 tell you? (2) What is an ablation study? (3) Name two reasons to prefer the simpler of two equal models.

CHAPTER 16

Turning the model into a website

Review 3 gives 10 marks for "end-to-end product development". Our product is a web page where anyone can test the model.

16.1 Saving the model

```
import pickle
with open("models/final_model.pkl", "wb") as f:      # "wb" = write binary
    pickle.dump({"model": final_model, "features": FEATURES}, f)

# later, to load it again:
with open("models/final_model.pkl", "rb") as f:
    saved = pickle.load(f)
saved["model"].predict(new_rows)
```

pickle saves a Python object to a file so you can load it again without retraining.

16.2 Why the model runs inside the browser

A website on GitHub Pages can only serve files; it cannot run Python. Logistic Regression is just "scale, multiply by weights, add, softmax", which is easy to redo in JavaScript. So we **export the numbers** to a small JSON file:

```
sc, clf = final_model.named_steps["sc"], final_model.named_steps["clf"]
export = {"mean": sc.mean_.tolist(), "scale": sc.scale_.tolist(),
          "coef": clf.coef_.tolist(), "intercept": clf.intercept_.tolist(),
          "features": FEATURES}
json.dump(export, open("app/model.json", "w"))
```

JSON is a text format for data (`{"key": value}`), readable by both Python and JavaScript.

16.3 The prediction in JavaScript

```
function predict(f) {
  const x  = MODEL.features.map(n => f[n]);                       // features in the right
order
  const xs = x.map((v, i) => (v - MODEL.mean[i]) / MODEL.scale[i]); // 1. scale
  const lg = MODEL.coef.map((w, k) =>                              // 2. weighted sum per
class
      w.reduce((s, wi, i) => s + wi * xs[i], MODEL.intercept[k]));
  const m = Math.max(...lg);
  const e = lg.map(v => Math.exp(v - m));                          // 3. softmax
  const t = e.reduce((a, b) => a + b, 0);
  return e.map(v => v / t);                                        // probabilities
}
```

This is exactly what scikit-learn does internally. The page also repeats the feature engineering of Chapter 8 in JavaScript (ratios, 14-day mean and SD, the 10% floor, z-scores).

Parity check. How do we know the JavaScript is right? We saved Python's probabilities for 15 real test days (`models/parity_check.json`) and compared them with the browser's. The biggest difference was 0.0005, i.e. identical apart from rounding.

16.4 How the page is built

The page is one HTML file with three parts:

- **HTML**: the structure (headings, buttons, input boxes, tables).
- **CSS**: the look (colours, fonts, layout, dark mode).
- **JavaScript**: the behaviour (read inputs → compute features → predict → draw bars).

`src/build_app.py` takes `app/template.html` and pastes in the model JSON and the results data, then writes `app/index.html` and `docs/index.html` :

```
html = template.replace("__MODEL__", json.dumps(model)).replace("__DASH__",
json.dumps(dash))
```

16.5 What the page does

Tab	Content
Predictor	Pick a real person and day from the test month (baseline = their previous 14 days), or type your own typical day and today. Shows the predicted label, the 3 probabilities, and where today sits against the personal baseline for each signal.
Data & cleaning	The audit table and the cleaning steps
Exploration	Class balance, weekday mix, personal baselines, raw vs z-score boxes, mutual information
Model comparison	All metrics, tuning, confusion matrices, clustering, ablation, importance
Final model	The winner and the reasons

Try it. Open the live app, choose U003 → Thu 18 Apr (an Unusual day). Then lower the social media and app switches until the prediction changes to Normal. You are watching the model's decision boundary.

CHAPTER 17

Putting it on GitHub and going live

17.1 Git and GitHub

- **Git** is a tool on your computer that saves snapshots (*commits*) of your project, so you can see history and undo mistakes.
- **GitHub** is a website that stores your Git project online (a *repository*, or *repo*) and can host websites.

17.2 Doing it yourself, step by step

Step 1. Install Git from `git-scm.com`. Then set your name once:

```
git config --global user.name "Jeshwanth"
git config --global user.email "your-email@example.com"
```

Step 2. Create an empty repository on github.com (green **New** button). Do not add a README.

Step 3. In your project folder:

```
git init                                    # start tracking this folder
git add .                                   # stage all files
git commit -m "First version of the project"   # save a snapshot
git branch -M main                          # name the branch "main"
git remote add origin https://github.com/Jeshwanth-tkd/Distraction-Pattern-Detector.git
git push -u origin main                     # upload
```

The first push opens a browser window to sign in to GitHub.

Step 4. Later changes are just three commands:

```
git add .
git commit -m "Describe what you changed"
git push
```

17.3 GitHub Pages: a free website

1. On the repo page: **Settings** → **Pages**.
2. Source: **Deploy from a branch**. Branch: **main**, folder: **/docs**. Click **Save**.
3. After about a minute your site is live at `https://jeshwanth-tkd.github.io/Distraction-Pattern-Detector/`.

GitHub Pages serves `docs/index.html` as the home page. That is why `build_app.py` writes the app into `docs/`.

17.4 The `.gitignore` file

Lists files Git should not upload (e.g. `__pycache__/`, temporary notebook checkpoints).

✓ **Check yourself.** What do `add`, `commit` and `push` each do?

CHAPTER 18

Preparing for your reviews

18.1 What to show in each review

Review	Show	Marks focus
1	Chapters 4–5: problem, objectives, dataset table, charter, methodology, timeline	Charter quality (10)
2	Notebook sections 2.1–2.6: audit table, cleaning log, validation, features, EDA charts, insights	EDA (10)
3	Sections 3.1–3.8 + live app: comparison table, tuning, selection with tests, website	Implementation (10), product (10)

18.2 A 2-minute explanation you can give

> *Raw screen time does not tell you if a day is unusual, because everyone's normal is different. My project builds a personal baseline from each person's previous 14 days and measures how far today is from it, as z-scores. I cleaned a dataset of 40 people over 120 days, fixing 10 kinds of data problems, and EDA showed that raw values overlap between classes but personal z-scores separate them. I compared six methods on a held-out month: a statistical z-score rule, Isolation Forest, K-Means, Logistic Regression, Decision Tree and Random Forest, tuned with grid search and person-grouped cross-validation. Logistic Regression and Random Forest tied statistically, so I chose the simpler Logistic Regression: 81% accuracy, 0.76 macro-F1, 0.90 ROC-AUC. It runs in a live web app hosted on GitHub Pages.*

18.3 Viva questions and answers

Q1. What is your problem statement? Classify a day of digital activity as Normal, Increased Distraction or Unusually Distracted relative to the person's own baseline.

Q2. Why is a personal baseline needed? Normal screen time varies from about 2.5 to 7.3 hours between people, so one threshold for all would always flag heavy users and never flag light users.

Q3. Where is the dataset from? Simulated with a fixed seed, because real labelled per-app logs are private. It models personal baselines, persistent distraction via a Markov chain, weekend effects and 7% label noise.

Q4. What data issues did you find? Missing values, 85 duplicates, inconsistent spellings, mixed date formats, negative minutes, totals over 24 h, night use over the night window, a 9999 glitch and totals not matching their parts.

Q5. How did you handle missing values? Impossible values became missing; all gaps were filled with the user's own median.

Q6. What is a z-score? (value − mean) ÷ standard deviation: how many SDs a day is from the person's usual level.

Q7. What is data leakage and how did you avoid it? Using information at training time that would not be available when predicting. The baseline uses `shift(1)` so only earlier days are used, the split is by time, and scaling is fitted inside pipelines on training data only.

Q8. Why split train and test by time? Because in use we always predict future days.

Q9. What is GroupKFold? Cross-validation where all days of one person stay in the same fold, so validation is always on unseen people.

Q10. Why macro-F1? Imbalanced classes; macro-F1 weights each class equally.

Q11. Precision vs recall? Precision: of the predicted positives, how many are right. Recall: of the real positives, how many were found.

Q12. What does ROC-AUC 0.898 mean? In about 90% of random pairs, the model scores the Unusual day higher than a non-Unusual day.

Q13. Why include a Z-score rule? As a transparent baseline that ML models must beat.

Q14. Why did Isolation Forest perform worst? It detects any unusual day, including unusually productive ones, so it confuses direction.

Q15. Why k = 3 in K-Means when k = 2 had a higher silhouette? The task has three states; k = 2 just splits normal from not-normal.

Q16. How does Logistic Regression make a prediction? Weighted sum of scaled features per class, then softmax into probabilities.

Q17. What does C control? Regularisation strength: smaller C keeps weights small for a simpler, more stable model.

Q18. Why did the single Decision Tree do badly at first? Unlimited depth overfits; limiting depth to 5 and leaves to 40 days improved it.

Q19. How is Random Forest better than one tree? It averages many trees trained on random samples of rows and features, which reduces variance.

Q20. How did you tune? GridSearchCV with GroupKFold, optimising macro-F1 on training data.

Q21. Why Logistic Regression over Random Forest? Statistically tied (McNemar p = 0.063, CI includes 0); LR is simpler, more stable, interpretable, calibrated and deployable in the browser.

Q22. What is McNemar's test? A test comparing two models on the same test items using only the items where they disagree.

Q23. What did the ablation show? Personal-baseline features add +0.069 macro-F1 to Logistic Regression.

Q24. What kind of errors remain? Almost all between neighbouring classes; only one Normal day was predicted Unusual.

Q25. How is the model deployed? Weights exported to JSON; JavaScript reproduces scaling and softmax in a static web page on GitHub Pages; verified against Python to within 0.0005.

Q26. Limitations? Simulated data, 7-day warm-up before a baseline exists, self-reported labels.

Q27. Future work? Validate on real consented logs, sequence models over consecutive days, per-user thresholds, on-device prediction for privacy.

Q28. Is this a medical tool? No. It flags changes in usage pattern only.

CHAPTER 19

Glossary

Term	Meaning
Ablation	Removing one part to measure its contribution
Accuracy	Share of predictions that are correct
Anomaly detection	Finding items unlike the rest
AUC	Area under the ROC curve; chance a positive is ranked above a negative
Baseline (model)	The simplest reasonable method, used as a reference
Baseline (personal)	A person's usual level, from their own previous 14 days
Bootstrap	Resampling the data with replacement to estimate uncertainty
Class imbalance	Some classes much more common than others
Classification	Predicting a category
Clustering	Grouping similar items without labels
Confusion matrix	Table of actual vs predicted classes
Cross-validation	Repeated train/check splits inside the training data
DataFrame	A pandas table
Data leakage	Information the model should not have sneaking into training
EDA	Exploratory data analysis
Encoding	Turning text categories into numbers
Ensemble	Combining many models
F1 score	Harmonic mean of precision and recall
Feature	An input column
Feature engineering	Creating new, more useful features
Gini impurity	How mixed the labels are in a group
Grid search	Trying every combination of hyperparameters
GroupKFold	Cross-validation that keeps each group (person) in one fold
Hyperparameter	A setting chosen before training
Imputation	Filling in missing values
IQR	Q3 − Q1, the width of the middle half of the data
Label / target	The answer to predict
Macro average	Plain average over classes

Term	Meaning
Markov chain	Next state depends only on the current state
Median	The middle value when sorted
Mutual information	How much a feature tells you about the label
Overfitting	Memorising training data, failing on new data
p-value	Chance of a result this extreme if there were no real effect
PCA	Projecting many features onto the 2 main directions of spread
Pipeline	A chain of steps (e.g. scale then model) fitted together
Precision	Of predicted positives, share that are correct
Recall	Of actual positives, share that were found
Regularisation	A penalty that keeps a model simple
Robust	Not much affected by extreme values
Seed	A number that makes random results repeatable
Silhouette score	How well each point fits its own cluster
Skewness	Lopsidedness of a distribution
Softmax	Turns scores into probabilities that sum to 1
Standard deviation	Typical distance of values from their mean
Supervised learning	Learning from labelled examples
Test set	Data held back to measure final performance
Unsupervised learning	Learning without labels
z-score	(value − mean) ÷ standard deviation